

Dataset Collection

Dataset Collection & Optimization:

Project Context:

This task is part of my AI internship project at BabiPoly, a West African digital board-games platform. The goal of the full machine learning project is to predict a player's engagement level based on their gameplay and behavioral data.

The target variable for the project is: EngagementLevel

The expected target classes are: Low, Medium, High

This task focuses on preparing the dataset and performing exploratory data analysis before moving into model building. The work will be completed in a Jupyter Notebook so that the code, outputs, plots, and explanations can all stay together in one reproducible file.

Task Objective:

The objective of Task 2 was to create a working Jupyter Notebook that performs the first stage of the machine learning workflow:

- 1- Load the online gaming behavior dataset
- 2- Audit the dataset for missing values and duplicate rows
- 3- Clean the dataset by handling missing values
- 4- Explore the target variable distribution
- 5- Visualize numeric feature distributions
- 6- Analyze correlations between numeric variables
- 7- Compare categorical features against the engagement target
- 8- Save a cleaned version of the dataset for later machine learning steps

The final output of this task is a cleaned CSV file that can be used in the next phase of the project.

This should be the structure of the project:

AI/github

├─ data/

| └─ raw/

| | └─ online_gaming_behavior.csv

```
| └─ cleaned_gaming.csv
└─ notebooks/
    | └─ engagement_prediction.ipynb
    └─ requirements.txt
    └─ README.md
```

The main notebook for this task is: notebooks/engagement_prediction.ipynb
Executed version after verification: notebooks/engagement_prediction_executed.ipynb
The cleaned dataset was saved as: data/cleaned_gaming.csv

Dataset Used:

The dataset used for this task is the Predict Online Gaming Behavior Dataset from Kaggle.

The dataset contains online gaming behavior records with player demographics, gameplay activity, and engagement labels.

Important columns include:

```
PlayerID
Age
Gender
Location
GameGenre
PlayTimeHours
InGamePurchases
GameDifficulty
SessionsPerWeek
AvgSessionDurationMinutes
PlayerLevel
AchievementsUnlocked
EngagementLevel
```

The dataset is intended to help predict whether a player has Low, Medium, or High engagement.

Environment Setup

Before working on the notebook, a Python virtual environment was created inside the project folder.

```
PS C:\Users\Administrator> cd C:\Users\Administrator\Desktop\Internship-2025\Internship-Projects\AI\github
PS C:\Users\Administrator\Desktop\Internship-2025\Internship-Projects\AI\github> py -3 -m venv .venv
>> .\.venv\Scripts\Activate.ps1
>> python -m pip install --upgrade pip
>> pip install jupyter pandas numpy matplotlib seaborn
>> python -m ipykernel install --user --name babipoly-gaming-ml --display-name "Python (BabiPoly Gaming ML)"
>> pip freeze > requirements.txt
```

```
.0 tornado-6.5.6 traitlets-5.15.0 typing-extensions-4.15.0 tzdata-2026.2 uri-template-1.3.0 urllib3-2.7.0 wcwidth-0.7.0 webcolors-25.10.0 webencod
extension-4.0.15
Installed kernelspec babipoly-gaming-ml in C:\Users\Administrator\AppData\Roaming\jupyter\kernels\babipoly-gaming-ml
(.venv) PS C:\Users\Administrator\Desktop\Internship-2025\Internship-Projects\AI\github>
```

pandas: Loading, cleaning, and analyzing tabular data

numpy: Numeric operations and selecting numeric columns

matplotlib: Creating charts and controlling plot layout

seaborn: Creating statistical visualizations

jupyter: Running the project as a notebook

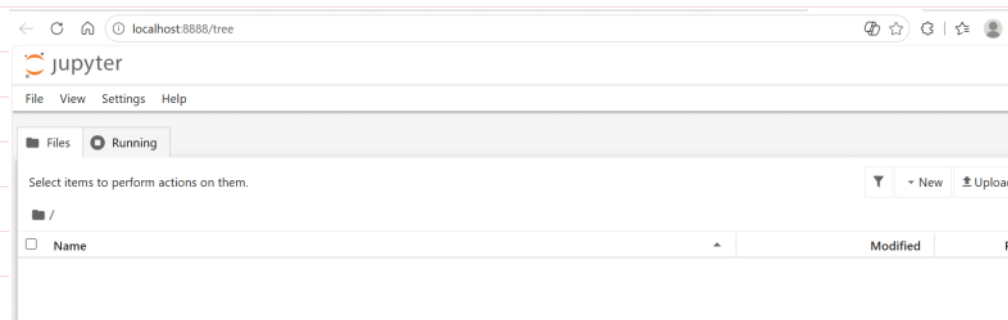
ipykernel: Connecting the virtual environment to Jupyter

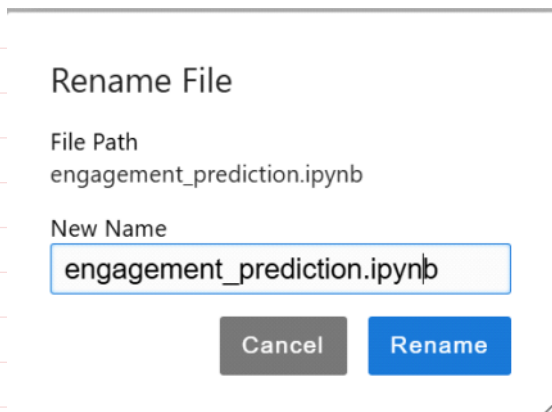
The notebook uses: %matplotlib inline

This makes plot appear directly inside the Jupyter Notebook output cells.

Creating the jupyter notebook:

```
(.venv) PS C:\Users\Administrator\Desktop\Internship-2025\Internship-Projects\AI\github\notebooks> jupyter notebook
```





Notebook Structure

The notebook was organized into seven code cells. Each cell represents one required step from the task prompt.

The seven notebook steps are:

- 1- Load data
- 2- Data quality audit
- 3- Target distribution
- 4- Numeric feature distributions
- 5- Correlation heatmap
- 6- Categorical features vs target
- 7- Save clean dataset

This structure makes the notebook easy to read, debug, and explain.

Step 1 - Load Data

The first notebook cell imports the required libraries and loads the dataset into a pandas DataFrame named: df

The notebook imports:

```
%matplotlib inline

from pathlib import Path
import math

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

The dataset is loaded from: data/raw/online_gaming_behavior_dataset.csv

The code uses "Path.cwd()" and "Path.cwd().parent" to find the project root. This makes the notebook more reliable because it can still find the dataset whether Jupyter is launched from the main project folder or from inside the notebooks folder.

The first cell prints:

```
print("Dataset shape:", df.shape)
print("\nData types:")
print(df.dtypes)
print("\nFirst 5 rows:")
df.head(5)
```

df.shape: Number of rows and columns

df.dtypes: Data type of each column

df.head(5): First five records in the dataset

Observed dataset shape during verification: (40036, 13)

```
Dataset shape: (40036, 13)***
```

This means the current CSV file has 40,036 rows and 13 columns.

	PlayerID	Age	Gender	Location	GameGenre	PlayTimeHours	InGamePurchases	GameDifficulty	SessionsPerWeek	AvgSessionDurationMinutes	Playe
0	9000	43	Male	Other	Strategy	16.27111876	0	Medium	6	108.0	
1	9001	29	Female	USA	Strategy	5.525961381	0	Medium	5	144.0	
2	9002	22	Female	USA	Sports	8.223755243	0	Easy	16	142.0	
3	9003	35	Male	USA	Action	5.265351277	1	Easy	9	85.0	
4	9004	33	Male	Europe	Action	15.53194452	0	Medium	2	131.0	

Step 2 - Data Quality Audit

The second notebook cell checks the quality of the data before analysis.

- ♦ It prints the number of missing values per column using: `df.isnull().sum()`
- ♦ It also checks for exact duplicate rows using: `df.duplicated().sum()`
- ♦ The cleaning logic then drops exact duplicate rows:
`df = df.drop_duplicates().reset_index(drop=True)`

After that, the notebook handles PlayerID separately before normal numeric and

categorical cleaning. This is important because PlayerID is an identifier, not a gameplay measurement. It should not be filled with the median.

The "PlayerID" cleaning rule is:

- 1- Convert "PlayerID" to numeric.
- 2- If a "PlayerID" is missing or contains invalid text, assign a new ID after the current maximum valid "PlayerID".
- 3- If the code cannot find any valid existing "PlayerID", delete the rows with invalid IDs.

In code, this is handled by converting invalid IDs to NaN, checking the maximum valid ID, and assigning replacement IDs in sequence:

```
print("Missing values before cleaning:")
missing_before = df.isnull().sum()
print(missing_before)

print("\nDuplicate rows before cleaning:", df.duplicated().sum())

df = df.drop_duplicates().reset_index(drop=True)

invalid_value_counts = {}

# PlayerID has special handling: repair invalid/missing IDs by continuing after the max valid ID.
if "PlayerID" in df.columns:
    original_player_id = df["PlayerID"].copy()
    df["PlayerID"] = pd.to_numeric(df["PlayerID"], errors="coerce")
    invalid_player_id_mask = df["PlayerID"].isna()
    invalid_value_counts["PlayerID"] = int(invalid_player_id_mask.sum())

    valid_player_ids = df.loc[~invalid_player_id_mask, "PlayerID"]
    if invalid_player_id_mask.any() and not valid_player_ids.empty:
        next_player_id = int(valid_player_ids.max()) + 1
        replacement_ids = range(next_player_id, next_player_id + int(invalid_player_id_mask.sum()))
        df.loc[invalid_player_id_mask, "PlayerID"] = list(replacement_ids)
        print("\nRepaired missing/invalid PlayerID values:")
        print(f"Assigned PlayerID values from {next_player_id} to {next_player_id + int(invalid_player_id_mask.sum()) - 1}")
    elif invalid_player_id_mask.any():
        df = df.loc[~invalid_player_id_mask].reset_index(drop=True)
        print("\nDropped rows with invalid PlayerID because no valid max PlayerID was available.")
```

After PlayerID is repaired, the notebook defines the remaining numeric columns and controlled categorical columns.

The numeric columns filled by median are:

```
Age
PlayTimeHours
InGamePurchases
SessionsPerWeek
AvgSessionDurationMinutes
PlayerLevel
AchievementsUnlocked
```

The controlled categorical columns are:

Gender
Location
GameGenre
GameDifficulty
EngagementLevel

For numeric columns, invalid text values are converted to missing values using `pd.to_numeric(..., errors="coerce")`. For categorical columns, the code checks each value against an allowed list.

Unexpected values are converted to missing values before imputation.

Missing Value Handling

The notebook handles missing and invalid values differently depending on the column type.

For PlayerID:

```
next_player_id = int(valid_player_ids.max()) + 1
replacement_ids = range(next_player_id, next_player_id + int(invalid_player_id_mask.sum()))
df.loc[invalid_player_id_mask, "PlayerID"] = list(replacement_ids)
```

This repairs missing or invalid player IDs by continuing the existing ID sequence. In the current dataset, the highest valid player ID before repair was 49033, so the two invalid player IDs were repaired as: 49034 49035

For numeric columns:

```
if column in df.columns:
    values_before = df[column].copy()
    df[column] = pd.to_numeric(df[column], errors="coerce")
    invalid_value_counts[column] = int(values_before.notna().sum() - df[column].notna().sum())

print(f"Number of columns fixed with median: ", end='')
for column in numeric_missing_columns:
    median_value = df[column].median()
    df[column] = df[column].fillna(median_value)
    print(f"{column}: {median_value}")
```

This converts invalid numeric values to missing values, then fills missing numeric values with the median of that column. The median is useful because it is less affected by extreme outliers than the mean.

For categorical columns:

```
if column in df.columns:
    df[column] = df[column].astype("string").str.strip()
    invalid_mask = df[column].notna() & ~df[column].isin(valid_values)
    invalid_value_counts[column] = int(invalid_mask.sum())
    df.loc[invalid_mask, column] = pd.NA

if categorical_missing_columns:
    print("\nCategorical columns filled with mode:")
    for column in categorical_missing_columns:
        mode_values = df[column].mode(dropna=True)
        mode_value = mode_values.iloc[0] if not mode_values.empty else "Unknown"
        df[column] = df[column].fillna(mode_value)
    print(f"{column}: {mode_value}")
```

This converts unexpected category labels to missing values, then fills missing categorical values with the most common value in that column. If a mode cannot be found, the code uses "Unknown" as a fallback.

The cell then confirms that no missing values remain:

```
print("\nDuplicate rows after cleaning:", df.duplicated().sum())
print("\nMissing values after cleaning:")
missing_after = df.isnull().sum()
print(missing_after)
print(f"\nClean summary: {missing_after.sum()} total null values remain.")
```

Data Quality Findings

During verification, the raw dataset contained:

- ♦ 14 total missing values
- ♦ 1 exact duplicate row
- ♦ 2 missing or invalid PayerID values

After running the cleaning step, the notebook confirmed:

- ♦ 0 total null values remain
- ♦ 0 exact duplicate rows remain

One important observation is that two rows near the end of the raw CSV appear to contain unusual or corrupted values. For example, one row has an unexpected target value such as: 9 instead of one of: {Low, Medium, High}

The base notebook code now treats that unexpected target value like a missing categorical value. It converts the invalid value to NaN, then replaces it with the most common valid value in the EngagementLevel column. In the current dataset, the mode is

Medium, so the invalid target is corrected to Medium during the cleaning step.

The same idea is also applied to normal numeric columns. If a numeric column contains invalid text, the value is converted to NaN and then replaced with the median of that numeric column.

PlayerID is handled differently because it is an identifier. Instead of filling it with the median, the notebook repaired the two invalid player IDs by assigning: 49034 49035

After filling values, the code checks for duplicates again. Any exact duplicates remaining after cleaning are dropped.

```
Missing values before cleaning:
```

```
PlayerID      1
Age           1
Gender        1
Location      0
GameGenre     1
PlayTimeHours 1
InGamePurchases 1
GameDifficulty 2
SessionsPerWeek 1
AvgSessionDurationMinutes 2
PlayerLevel   1
AchievementsUnlocked 1
EngagementLevel 1
dtype: int64
```

```
Duplicate rows before cleaning: 1
```

```
Repaired missing/invalid PlayerID values:
```

```
Assigned PlayerID values from 49034 to 49035
```

```
Invalid values converted to missing:
```

```
PlayerID: 2
Age: 1
PlayTimeHours: 1
InGamePurchases: 1
SessionsPerWeek: 1
PlayerLevel: 1
AchievementsUnlocked: 1
Gender: 1
Location: 2
GameGenre: 1
```

```
Numeric columns filled with median:
```

```
Age: 32.0
PlayTimeHours: 12.008002155
InGamePurchases: 0.0
SessionsPerWeek: 9.0
AvgSessionDurationMinutes: 95.0
PlayerLevel: 49.0
AchievementsUnlocked: 25.0
```

```
Categorical columns filled with mode:
```

```
Gender: Male
Location: USA
GameGenre: Sports
GameDifficulty: Easy
EngagementLevel: Medium
```

```
Duplicate rows after cleaning: 0
```

```
Missing values after cleaning:
```

```
PlayerID      0
Age           0
Gender        0
Location      0
GameGenre     0
PlayTimeHours 0
InGamePurchases 0
GameDifficulty 0
SessionsPerWeek 0
AvgSessionDurationMinutes 0
PlayerLevel   0
AchievementsUnlocked 0
EngagementLevel 0
dtype: int64
```

Clean summary: 0 total null values remain.
Cleaned dataset shape after duplicate removal: (40036, 13)

EngagementLevel values after cleaning:
EngagementLevel
Medium 19376
High 10336
Low 10324
Name: count, dtype: Int64

Last 5 PlayerID values after cleaning:
[49031, 49032, 49033, 49034, 49035]

The initial dataset:

49032	34 Male	USA	Sports	14.01782	1 Medium	3	128	70	10 Medium
49033	19 Male	USA	Sports	10.0838	0 Easy	13	84	72	39 Medium
49033	19 Male	USA	Sports	10.0838	0 Easy	13	84	72	39 Medium

The cleaned dataset:

49032	34 Male	USA	Sports	14.01782	1 Medium	3	128	70	10 Medium
49033	19 Male	USA	Sports	10.0838	0 Easy	13	84	72	39 Medium
49034	32 Male	USA	Sports	12.008	0 Easy	9	95	49	25 Medium
49035	32 Male	USA	Sports	12.008	0 Easy	9	95	49	25 Medium

Step 3 - Target Distribution

The third notebook cell analyzes the target column: EngagementLevel

It calculates the class proportions using:

```
target_counts = df["EngagementLevel"].value_counts()  
target_percentages = df["EngagementLevel"].value_counts(normalize=True)
```

It also creates a bar chart showing the count of each engagement class.

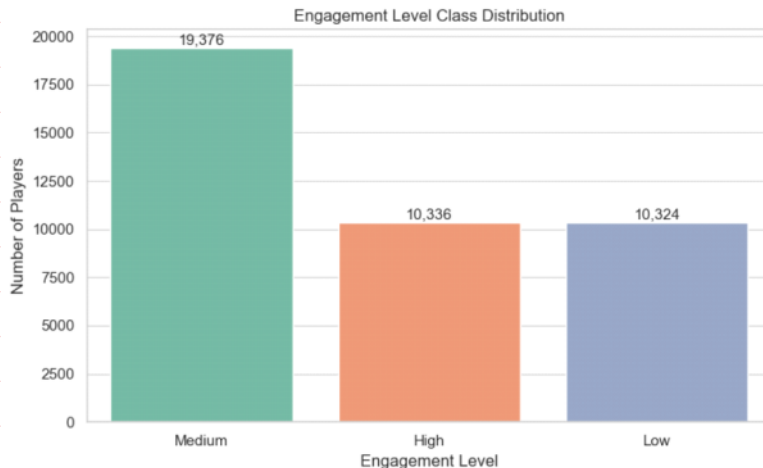
The chart includes:

Chart Element	Description
X-axis	Engagement level classes
Y-axis	Number of players
Bar labels	Exact count for each class
Title	Engagement Level Class Distribution

Observed target distribution after cleaning:

Engagement Level	Count	Percentage
Medium	19,376	48.39%
High	10,336	25.82%
Low	10,324	25.79%

```
EngagementLevel distribution:
EngagementLevel
Medium    0.483964
High      0.258168
Low       0.257868
Name: proportion, dtype: Float64
```



Balance Comment

The dataset is not perfectly balanced because the Medium class is much larger than both High and Low.

However, High and Low are very close to each other in size. The main imbalance is that Medium represents almost half of the dataset.

This matters for machine learning because a model may learn to predict Medium more often than the other classes. During model training, it may be useful to check classification metrics such as precision, recall, F1-score, and confusion matrix instead of relying only on accuracy.

```
max_share = target_percentages.max()
min_share = target_percentages.min()

if max_share - min_share <= 0.10:
    balance_comment = "The dataset appears reasonably balanced because the class percentages are close to each other."
else:
    balance_comment = "The dataset appears imbalanced because at least one class is much more common than another."

print("Balance comment:", balance_comment)
```

Balance comment: The dataset appears imbalanced because at least one class is much more common than another.

Step 4 - Numeric Feature Distributions

The fourth notebook cell plots histograms for all numeric columns.

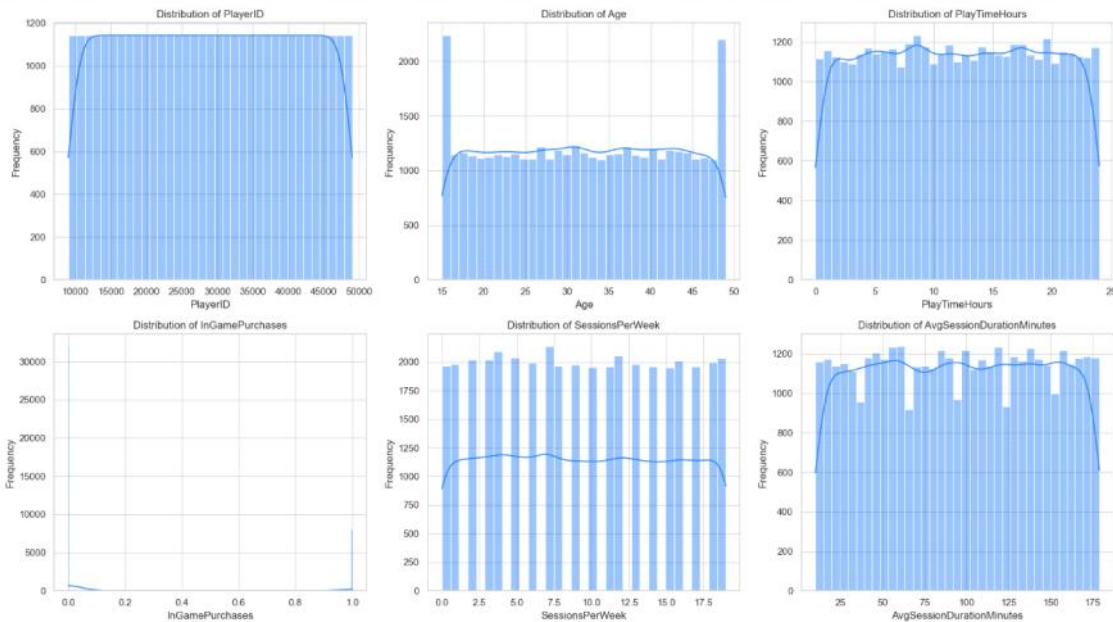
The code uses:

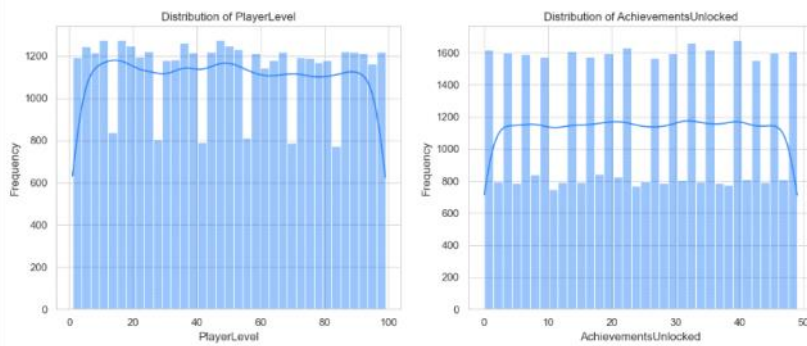
```
for index, column in enumerate(numeric_columns):
    sns.histplot(data=df, x=column, kde=True, color="#2F80ED")
    axes[index].set_title(f"Distribution of {column}")
    axes[index].set_xlabel(column)
    axes[index].set_ylabel("Frequency")
```

Each histogram includes:

Element	Purpose
Histogram bars	Show how values are distributed
KDE curve	Smooth curve estimating the distribution shape
Plot title	Shows the column name
X-axis label	Shows the feature being plotted
Y-axis label	Shows frequency

The plots are arranged in a grid with three columns per row.





This step helps identify:

- ♦ Whether numeric values are normally distributed or skewed.
- ♦ Whether there are possible outliers.
- ♦ Whether some features have concentrated or spread-out values.
- ♦ Whether scaling may be needed before machine learning.

Step 5 - Correlation Heatmap

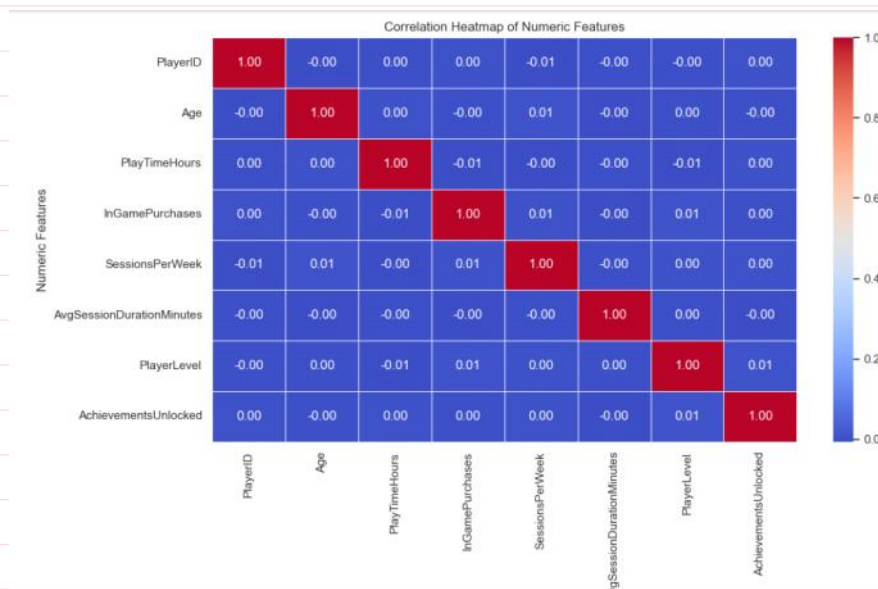
The fifth notebook cell creates a correlation heatmap for numeric columns.

The correlation matrix is computed using:

```
numeric_df = df.select_dtypes(include=[np.number])
correlation_matrix = numeric_df.corr()
```

The heatmap is plotted using:

```
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap of Numeric Features")
```



The heatmap includes:

Setting	Meaning
<code>annot=True</code>	Shows correlation numbers inside the cells
<code>cmap="coolwarm"</code>	Uses blue/red colors for negative/positive relationships
<code>fmt=".2f"</code>	Rounds correlation values to two decimals
<code>linewidths=0.5</code>	Adds spacing between cells

This step helps identify relationships between numeric features.

Correlation values are interpreted as:

Correlation Value	Meaning
Close to 1	Strong positive relationship
Close to -1	Strong negative relationship
Close to 0	Weak or no linear relationship

For example, if "SessionsPerWeek" and "" are strongly correlated, both features may be measuring similar behavior. This can matter later during feature selection.

Step 6 - Categorical Features vs Target

The sixth notebook cell compares categorical features against "EngagementLevel".

The required categorical features were:

- ♦ Gender
- ♦ Location
- ♦ GameGenre

For each feature, the notebook creates a grouped countplot:

```
for column in categorical_features:
    plt.figure(figsize=(12, 6))
    ax = sns.countplot(data=df, x=column, hue="EngagementLevel")
    plt.title(f"{column} Count by Engagement Level")
```

Each chart shows how the counts of Low, Medium, and High engagement differ across categories.

The notebook also rotates the x-axis labels:

```
plt.xlabel("Number of Players")
plt.xticks(rotation=45, ha="right")
plt.legend(title="Engagement Level")
plt.tight_layout()
```

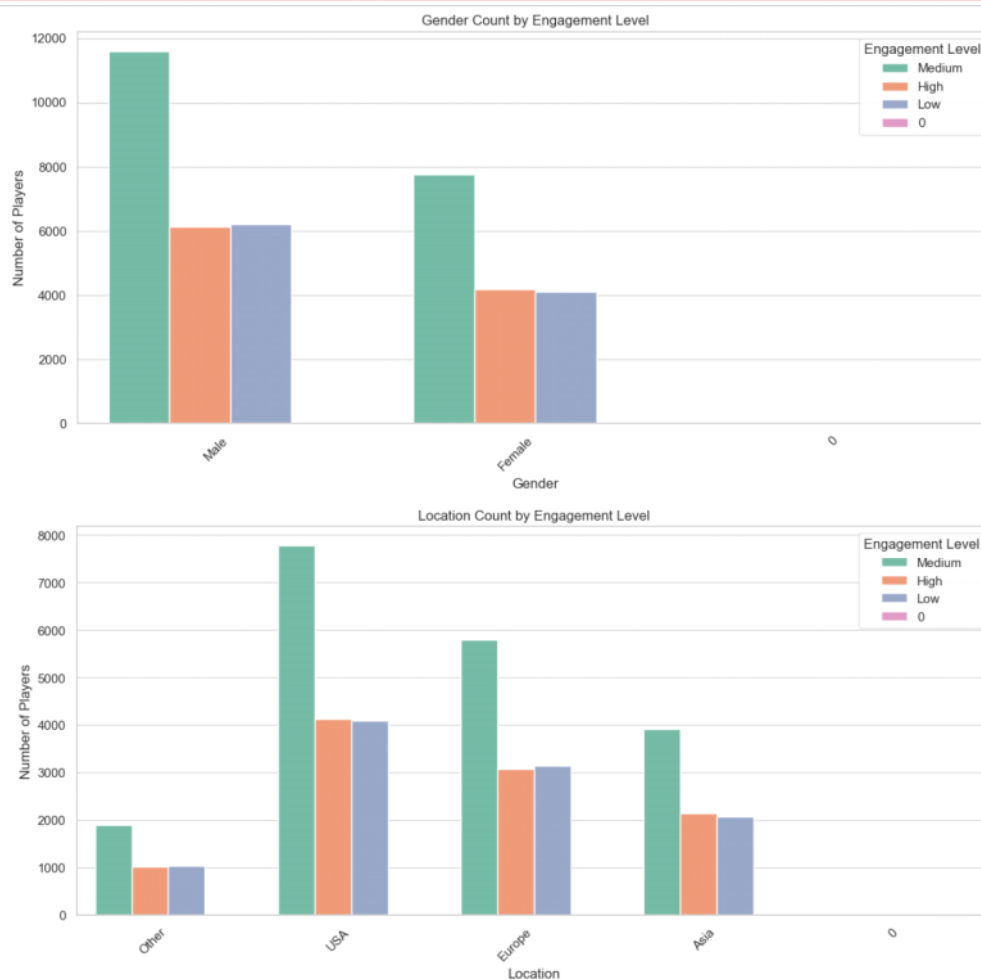
This improves readability, especially when categories have longer names.

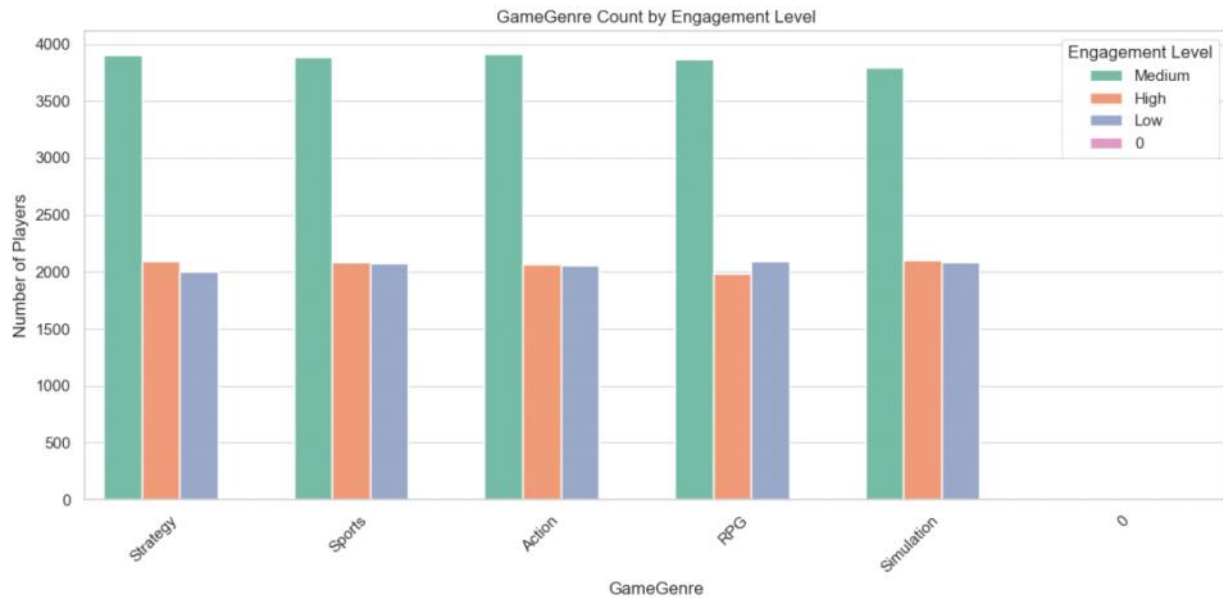
Purpose of This Step

This step helps answer questions such as:

- 1- Do some game genres have more highly engaged players?
- 2- Are engagement patterns different across locations?
- 3- Is engagement similar across gender groups?
- 4- Are some categories dominated by one engagement level?

These insights can help decide which categorical variables may be useful when building the final prediction model.





Step 7 - Save Clean Dataset

The final notebook cell saves the cleaned DataFrame to a CSV file.

The code creates the folder if needed, Then it saves the cleaned DataFrame:

```
clean_data_path = project_root / "data" / "cleaned_gaming.csv"
clean_data_path.parent.mkdir(parents=True, exist_ok=True)

df.to_csv(clean_data_path, index=False)

print(f"Dataset saved. Shape: {df.shape}")
print(f"Saved to: {clean_data_path}")
```

Dataset saved. Shape: (40036, 13)

Issues Encountered and Fixes Applied

During testing, the second notebook cell needed stronger data-quality handling. Some rows contained invalid text in numeric fields, unexpected labels in categorical fields, and invalid or missing "PlayerID" values.

The cleaning logic was improved so that:

- 1- PlayerID is repaired by assigning the next available IDs after the maximum valid ID.
- 2- Rows with invalid PlayerID values are deleted only if no valid existing PlayerID is available.
- 3- Normal numeric columns convert invalid text to NaN and then fill with the median.
- 4- Controlled categorical columns convert unexpected labels to NaN and then fill with the mode.

5- Exact duplicates are checked again after cleaning.

The executed notebook confirmed this specific PlayerID repair:

```
Repaired missing/invalid PlayerID values:  
Assigned PlayerID values from 49034 to 49035
```

This means the corrupted rows were preserved where possible instead of being removed, while still keeping PlayerID numeric and unique.

Featur Engineering

Before starting Task 3, I already have the cleaned dataset from Task 2. That file comes from loading, auditing, cleaning, exploring, and saving the original gaming dataset: `data/cleaned_gaming.csv`

Task 3 continues from that cleaned dataset and prepares the data for machine learning model training. This step is important because raw cleaned data is still not always ready for a model. Machine learning algorithms usually need:

- 1- Numeric target labels instead of text categories.
- 2- Engineered features that better represent player behavior.
- 3- Encoded categorical columns.
- 4- Scaled numeric values.
- 5- A selected subset of useful features.
- 6- Train and test files that I save for the modeling stage.

The target variable for the project is: `EngagementLevel`

The target classes are: `{Low, Medium, High}`

For modeling, I convert these text labels into numeric labels:

- ♦ Low --> 0
- ♦ Medium --> 1
- ♦ High --> 2

Task Objective

The objective of Task 3 is to create a complete feature engineering pipeline that starts from the cleaned dataset and produces training and testing files for the next machine learning task.

The required steps I am following are:

- 1- Load `data/cleaned_gaming.csv`.
- 2- Print the dataset columns and shape.
- 3- Encode the `EngagementLevel` target into a numeric target column.
- 4- Create three new engineered features.
- 5- One-hot encode selected categorical columns.

- 6- Scale numeric features using StandardScaler.
- 7- Score features with SelectKBest and f_classif.
- 8- Select the top 14 features.
- 9- Split the data into training and testing sets.
- 10- Save the split datasets and selected feature list into the data/ folder.

The final output of this task is a set of CSV artifacts that I can use directly for model training.

Dataset Used

For this task, I use the cleaned version of the online gaming behavior dataset from Task 2.

The cleaned dataset contains: 40036 rows, 13 columns

Important feature categories:

Column	Meaning
PlayerID	Unique player identifier
Age	Player age
Gender	Player gender
Location	Player region
GameGenre	Type of game played
PlayTimeHours	Total play time
InGamePurchases	Whether the player made purchases
GameDifficulty	Difficulty level
SessionsPerWeek	Number of sessions per week
AvgSessionDurationMinutes	Average session duration
PlayerLevel	Current player level
AchievementsUnlocked	Number of achievements unlocked
EngagementLevel	Target class

Before encoding the target, the dataset target distribution is:

Medium: 19376

High: 10336

Low: 10324

This means the Medium class is the largest class, while Low and High are nearly balanced with each other.

Environment Setup

The project uses a Python virtual environment inside the github folder: `.venv`

I run the feature engineering script using:

```
.\venv\Scripts\python.exe feature_engineering_pipeline.py
```

The main libraries I use in Task 3 are:

Library	Purpose
pandas	Load CSV files, create new columns, encode categorical columns, and save artifacts
numpy	Select numeric columns
scikit-learn	Scale features, score features, select features, and split train/test data

The specific sklearn tools I use are:

Tool	Purpose
<code>StandardScaler</code>	Standardize numeric columns
<code>SelectKBest</code>	Select the strongest features
<code>f_classif</code>	Score features using ANOVA F-values for classification
<code>train_test_split</code>	Split the selected features into train and test datasets

While working on the pipeline, I notice that scikit-learn is missing from the virtual environment.

I install it into the project environment and update the dependency list in: `requirements.txt`

Relevant packages now include:

- ♦ `scikit-learn==1.9.0`
- ♦ `scipy==1.17.1`
- ♦ `joblib==1.5.3`
- ♦ `threadpoolctl==3.6.0`
- ♦ `narwhals==2.22.0`

Pipeline Structure

I implement the pipeline in one Python script: `feature_engineering_pipeline.py`

The script is organized into eight required steps:

- 1- Load data.
- 2- Encode the target.
- 3- Create engineered features.
- 4- Encode categorical features.
- 5- Scale numeric features.
- 6- Select features.
- 7- Split train and test data.
- 8- Save artifacts.

This structure follows the task requirements exactly and makes the script easy to rerun.

Step 1 - Load Data

The first step loads the cleaned dataset from: data/cleaned_gaming.csv

The code uses Path("data") so the file paths are clear and reusable:

```
# STEP 1 - Load Data
# Load the cleaned dataset created in Task 2 and inspect its structure.
data_dir = Path("data")
cleaned_data_path = data_dir / "cleaned_gaming.csv"

df = pd.read_csv(cleaned_data_path)
```

After loading the CSV, I print:

```
print("STEP 1 - df.columns:")
print(df.columns)
print("\nSTEP 1 - df.shape:")
print(df.shape)
```

This verifies that the cleaned dataset is loading correctly before any transformations are applied.

Output I get

```
STEP 1 - df.columns:
Index(['PlayerID', 'Age', 'Gender', 'Location', 'GameGenre', 'PlayTimeHours',
      'InGamePurchases', 'GameDifficulty', 'SessionsPerWeek',
      'AvgSessionDurationMinutes', 'PlayerLevel', 'AchievementsUnlocked',
      'EngagementLevel'],
      dtype='str')

STEP 1 - df.shape:
(40036, 13)
```

Step 2 - Encode the Target

The target column in the cleaned dataset is: EngagementLevel

Since machine learning models work better with numeric labels, I map the text classes into numbers:

```
# Convert EngagementLevel into numeric class labels for machine Learning.
target_mapping = {"Low": 0, "Medium": 1, "High": 2}
df["target"] = df["EngagementLevel"].map(target_mapping)
```

The new encoded target column is: target

The script also checks for unmapped labels:

```
target_mapping = {"Low": 0, "Medium": 1, "High": 2}
df["target"] = df["EngagementLevel"].map(target_mapping)

if df["target"].isnull().any():
    missing_targets = df.loc[df["target"].isnull(), "EngagementLevel"].unique()
    raise ValueError(f"Unmapped EngagementLevel values found: {missing_targets}")
```

This prevents silent errors if the dataset contains an unexpected label.

Encoded target distribution I get:

```
STEP 2 - Encoded target value counts:
target
0      10324
1      19376
2      10336
Name: count, dtype: int64
```

Step 3 - Create Engineered Features

I create three new engineered features to capture player behavior more meaningfully than the original raw columns alone.

The new features are:

- ♦ WeeklyPlayLoad
- ♦ AchievementRate
- ♦ SessionDepth

Feature 1 - WeeklyPlayLoad

Formula: $\text{WeeklyPlayLoad} = \text{SessionsPerWeek} * \text{AvgSessionDurationMinutes}$

```
# SessionDepth: average session depth adjusted by weekly session frequency
df["WeeklyPlayLoad"] = (
    df["SessionsPerWeek"] * df["AvgSessionDurationMinutes"]
)
```

Purpose?

This feature estimates total weekly gameplay minutes for a player.

Example, if a player has:

- ♦ SessionsPerWeek = 6

- ♦ $\text{AvgSessionDurationMinutes} = 108$

Then $\text{WeeklyPlayLoad} = 6 * 108 = 648$ weekly minutes

This feature is useful because total weekly activity is likely related to engagement level.

Feature 2 - AchievementRate

Formula: $\text{AchievementRate} = \text{AchievementsUnlocked} / (\text{PlayerLevel} + 1)$

```
df["WeeklyPlayLoad"] = (  
    df["SessionsPerWeek"] * df["AvgSessionDurationMinutes"]  
)  
df["AchievementRate"] = df["AchievementsUnlocked"] / (df["PlayerLevel"] + 1)
```

Purpose?

This feature measures how many achievements a player unlocks relative to their level.

The formula uses: $\text{PlayerLevel} + 1$

instead of only "PlayerLevel" to avoid division by zero and to make the calculation stable.

This feature can help distinguish players who progress efficiently from players who have high levels but fewer achievements.

Feature 3 - SessionDepth

Formula: $\text{SessionDepth} = \text{AvgSessionDurationMinutes} / (\text{SessionsPerWeek} + 1)$

```
df["AchievementRate"] = df["AchievementsUnlocked"] / (df["PlayerLevel"] + 1)  
df["SessionDepth"] = (  
    df["AvgSessionDurationMinutes"] / (df["SessionsPerWeek"] + 1)  
)
```

Purpose?

This feature measures average session depth while adjusting for weekly session frequency.

Also the formula uses: $\text{SessionPerWeek} + 1$, to avoid division by zero if a player has zero weekly sessions

This feature helps represent the difference between:

- ♦ Players who play often in shorter sessions
- ♦ Players who play less often but spend longer time per session

Engineered Feature Summary

After creating the three features, I print:

```
print("\nSTEP 3 - Engineered feature summary:")
print(df[["WeeklyPlayLoad", "AchievementRate", "SessionDepth"]].describe())
```

The output I get:

```
STEP 3 - Engineered feature summary:
      WeeklyPlayLoad  AchievementRate  SessionDepth
count      40036.000000      40036.000000  40036.000000
mean         897.673544         1.037441    17.044842
std          771.261230         2.003945    24.769212
min           0.000000         0.000000     0.500000
25%          268.000000         0.236111     5.000000
50%          684.000000         0.486842     9.000000
75%         1364.000000         0.941459    17.250000
max          3401.000000        24.500000   179.000000
```

Important observations:

Feature	Observation
WeeklyPlayLoad	Average weekly load is about 898 minutes
AchievementRate	Median achievement rate is about 0.49
SessionDepth	Median session depth is 9 minutes
WeeklyPlayLoad max	Some players have very high weekly play load
SessionDepth max	Some players have very long session depth values

Later in feature selection, these engineered features receive scores, and all three appear in the selected feature set.

Step 4 - Encode Categorical Features

The task required one-hot encoding for these categorical columns:

- ♦ Gender
- ♦ Location
- ♦ GameGenre

```
categorical_columns = ["Gender", "Location", "GameGenre"]
df = pd.get_dummies(df, columns=categorical_columns, drop_first=True)

print("\nSTEP 4 - Column list after one-hot encoding:")
print(df.columns.tolist())
```

The option `drop_first=True`, removes the first category from each encoded group. This helps reduce duplicate info because one category can be inferred from the others.

For example, if "Gender" has two values, one dummy column is enough: Gender_Male

If Gender_Male is False, he record belongs to the dropped reference category.

Column list I get after one-hot encoding:

```
STEP 4 - Column list after one-hot encoding:  
['PlayerID', 'Age', 'PlayTimeHours', 'InGamePurchases', 'GameDifficulty', 'SessionsPerWeek', 'AvgSessionDurationMinutes', 'PlayerLevel', 'AchievementsUnlocked', 'EngagementLevel', 'target', 'WeeklyPlayLoad', 'AchievementRate', 'SessionDepth', 'Gender_Male', 'Location_Europe', 'Location_Other', 'Location_USA', 'GameGenre_RPG', 'GameGenre_Simulation', 'GameGenre_Sports', 'GameGenre_Strategy']
```

New encoded categorical columns:

- ♦ Gender_Male
- ♦ Location_Europe
- ♦ Location_Other
- ♦ Location_USA
- ♦ GameGenre_RPG
- ♦ GameGenre_Simulation
- ♦ GameGenre_Sports
- ♦ GameGenre_Strategy

Step 5 - Scale Numeric Features

Numeric features can have very different ranges.

Example:

- ♦ Age : usually tens
- ♦ PlayTimeHours: can be much larger
- ♦ WeeklyPlayLoad: can reach thousands of minutes
- ♦ AchievementRate: Usually small decimal values

To make numeric features comparable, the script applies: StandardScaler()

The scaler transforms numeric columns so they are centered around mean 0 with standard deviation 1.

The script intentionally excludes:

```
excluded_from_scaling = {"target"}  
encoded_categorical_columns = [  
    column  
    for column in df.columns  
    if column.startswith(("Gender_", "Location_", "GameGenre_"))  
]
```

Reason?

- 1- "target" should never be scaled because it is the label the model will predict
- 2- One-hot encoded columns are already binary indicators and should remain as 0/1 or boolean values

```

excluded_from_scaling = {"target"}
encoded_categorical_columns = [
    column
    for column in df.columns
    if column.startswith(("Gender_", "Location_", "GameGenre_"))
]

numeric_columns = [
    column
    for column in df.select_dtypes(include=[np.number]).columns
    if column not in excluded_from_scaling
    and column not in encoded_categorical_columns
]

scaler = StandardScaler()
df[numeric_columns] = scaler.fit_transform(df[numeric_columns])

print("\nSTEP 5 - Numeric columns scaled:")
print(numeric_columns)

```

STEP 5 - Numeric columns scaled:
 ['PlayerID', 'Age', 'PlayTimeHours', 'InGamePurchases', 'SessionsPerWeek', 'AvgSessionDurationMinutes', 'PlayerLevel', 'AchievementsUnlocked', 'WeeklyPlayLoad', 'AchievementRate', 'SessionDepth']

Step 6 - Feature Selection

I perform feature selection to choose the strongest features before model training.

The target is: `y = df["target"].astype(int)`

I build the feature matrix by dropping:

- ♦ target
- ♦ EngagementLevel

```

columns_to_drop_from_x = ["target", "EngagementLevel"]
X = df.drop(columns=columns_to_drop_from_x)
X = X.select_dtypes(include=[np.number, "bool"])
y = df["target"].astype(int)

```

Dropped Column	Reason
target	It is the encoded label
EngagementLevel	It is the original text label and would leak the answer
GameDifficulty	It remains text and is excluded because only numeric and boolean columns are selected

For feature scoring, I use:

`SelectKBest(score_func=f_classif, k="all")`

The scoring function: `f_classif` , calculates ANOVA F-values between each feature and the classification target.
Higher scores mean the feature has stronger statistical separation across the target classes.

```
selector = SelectKBest(score_func=f_classif, k="all")
selector.fit(X, y)

feature_scores = pd.DataFrame(
    {
        "feature": X.columns,
        "score": selector.scores_,
    }
).sort_values(by="score", ascending=False)
```

Ranked feature scores I get:

```
STEP 6 - Feature scores ranked descending:
      feature      score
WeeklyPlayLoad 35562.282541
SessionsPerWeek 11626.388770
AvgSessionDurationMinutes 6207.623323
SessionDepth 3023.623647
AchievementsUnlocked 117.084043
PlayerLevel 108.127547
AchievementRate 11.823539
GameGenre_RPG 1.993712
PlayTimeHours 1.824019
GameGenre_Simulation 1.673896
Age 1.507421
GameGenre_Strategy 1.496817
InGamePurchases 1.493519
Location_Europe 0.777238
Gender_Male 0.594103
Location_USA 0.521365
PlayerID 0.387822
Location_Other 0.335653
GameGenre_Sports 0.009176
```

The top scoring feature is: `WeeklyPlayLoad`.

This makes sense because weekly gameplay load combines both session frequency and average session duration, which are likely strong indicators of engagement.

The next strongest features are:

- ♦ `SessionsPerWeek`
- ♦ `AvgSessionDurationMinutes`
- ♦ `SessionDepth`

This shows that gameplay frequency and session behavior are very important for predicting engagement level.

I select the top 14 features:

```
top_k = 14
selected_feature_names = feature_scores.head(top_k)["feature"].tolist()
X_selected = X[selected_feature_names]
```

Selected top 14 features:

```
STEP 6 - Selected top 14 features:
['WeeklyPlayLoad', 'SessionsPerWeek', 'AvgSessionDurationMinutes', 'SessionDepth', 'AchievementsUnlocked', 'PlayerLevel', 'AchievementRate', 'GameGenre_RPG', 'PlayTimeHours', 'GameGenre_Simulation', 'Age', 'GameGenre_Strategy', 'InGamePurchases', 'Location_Europe']
```

Step 7 - Train/Test Split

After selecting the top 14 features, I split the dataset into training and testing sets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_selected,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y,
)

print("\nSTEP 7 - X_train shape:")
print(X_train.shape)
print("\nSTEP 7 - X_test shape:")
print(X_test.shape)
print("\nSTEP 7 - Class distribution in y_train:")
print(y_train.value_counts().sort_index())
```

The split settings are:

Setting	Value	Purpose
test_size	0.20	Use 20 percent of the data for testing
random_state	42	Make the split reproducible
stratify	y	Preserve class distribution in train and test sets

Using "stratify=y" is important because the target classes are not perfectly balanced. Stratification makes sure the train and test sets keep a similar class distribution

Split output I get:

```
STEP 7 - X_train shape:
(32028, 14)

STEP 7 - X_test shape:
(8008, 14)

STEP 7 - Class distribution in y_train:
target
0      8259
1     15500
2      8269
Name: count, dtype: int64
```

Final split summary:

File	Rows	Columns
X_train.csv	32,028	14
X_test.csv	8,008	14
y_train.csv	32,028	1
y_test.csv	8,008	1

The 80/20 split gives me:

- ♦ 32028 testing rows
- ♦ 8008 testing rows

- ◆ 40036 total rows

Step 8 - Save Artifacts

The final step saves the selected training and testing data to the data/ folder.

```
X_train.to_csv(data_dir / "X_train.csv", index=False)
X_test.to_csv(data_dir / "X_test.csv", index=False)
y_train.to_csv(data_dir / "y_train.csv", index=False, header=["target"])
y_test.to_csv(data_dir / "y_test.csv", index=False, header=["target"])

with open(data_dir / "selected_features.txt", "w", encoding="utf-8") as file:
    for feature_name in selected_feature_names:
        file.write(f"{feature_name}\n")

print(
    f"\nFeature engineering complete. {len(X_selected.columns)} features selected."
)
```

Feature engineering complete. 14 features selected.

[]:

cleaned_gaming	Microsoft Excel Com...	3,033 KB
selected_features	TXT File	1 KB
X_test	Microsoft Excel Com...	1,721 KB
X_train	Microsoft Excel Com...	6,883 KB
y_test	Microsoft Excel Com...	24 KB
y_train	Microsoft Excel Com...	94 KB

Issues :

Issue 1:

During the first quick dataset check, I see that the system Python environment does not have pandas installed.

Issue: ModuleNotFoundError: No module named 'pandas'

Fix: the project already had a local virtual environment: .venv

So I run the script and checks using the project env:

.\venv\Scripts\python.exe

Issue 2:

The original target column: EngagementLevel contains the answer in text form.

The encoded target column: target also contains the answer

Fix: I exclude both columns from the feature matrix.

This prevents the model from accidentally learning directly from the answer column.

At this point, Task 3 is complete.

The project now has a complete feature engineering pipeline that:

- 1- Loads the cleaned gaming dataset.
- 2- Encodes the engagement target.
- 3- Creates three behavioral engineered features.
- 4- One-hot encodes the requested categorical variables.
- 5- Scales numeric features.
- 6- Scores all model-ready features.
- 7- Selects the top 14 features.
- 8- Splits the dataset into train and test sets.
- 9- Saves all required artifacts for modeling.

Model Training

Task Objective:

The objective of Task 4 is to train four classification models, compare them using 5-fold cross-validation on the training set, fit each model on the full training data, visualize the cross-validation results, inspect Random Forest feature importance, and save all trained models into the models/ folder.

The required models I am training are:

- 1- Logistic Regression
- 2- Decision Tree
- 3- Random Forest
- 4- Neural Network using MLPClassifier

The required evaluation metric for cross-validation is: f1_weighted

I use weighted F1 because this is a multiclass classification problem, and the class counts are not perfectly equal.

Weighted F1 gives me a more useful overall score because it considers precision and recall while also respecting the number of examples in each class.

Step 1 - Imports

I start by importing the required machine learning models from scikit-learn.

The requested classifier imports are:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.exceptions import ConvergenceWarning
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.neural_network import MLPClassifier
from sklearn.tree import DecisionTreeClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)
```

I also import the cross-validation tools:

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
```

I also use:

```

from pathlib import Path
import re
import time

import joblib
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

```

For saving the trained models, I import: `import joblib`

I set Matplotlib to use the Agg backend, because I want the script to save plot images directly without waiting for an interactive chart window.

Loading the Train/Test Splits

After the imports, I define the folders:

```

DATA_DIR = Path("data")
MODELS_DIR = Path("models")
PLOTS_DIR = Path("plots")

```

Then I load the data:

```

X_train = pd.read_csv(DATA_DIR / "X_train.csv")
X_test = pd.read_csv(DATA_DIR / "X_test.csv")
y_train = pd.read_csv(DATA_DIR / "y_train.csv").squeeze("columns")
y_test = pd.read_csv(DATA_DIR / "y_test.csv").squeeze("columns")

```

I use `.squeeze("columns")` for `y_train` and `y_test` because these CSV files contain one column named target.

The model expects the target as a one-dimensional series, not as a two-dimensional dataframe.

Step 2 - Define Models

I create a dictionary called `MODELS` so I can loop through all models using the same workflow.

```

MODELS = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree": DecisionTreeClassifier(max_depth=10, random_state=42),
    "Random Forest": RandomForestClassifier(
        n_estimators=200, random_state=42, n_jobs=-1
    ),
    "Neural Network (MLP)": MLPClassifier(
        hidden_layer_sizes=(128, 64), max_iter=300, random_state=42
    ),
}

```

I keep `random_state=42` wherever the model supports it so the results stay reproducible.

Why I use each model:

- 1- Logistic Regression gives me a simple linear baseline.
- 2- Decision Tree gives me an interpretable non-linear model.

- 3- Random Forest gives me an ensemble model that usually performs strongly on tabular data.
- 4- MLP gives me a neural-network approach for comparison.

Step 3 - Cross-Validation on the Training Set

I use 5-fold stratified cross-validation:

```
cv = StratifiedKFold(n_splits=5)
```

I use StratifiedKFold because I want each fold to keep a similar class distribution. This is important for multiclass engagement prediction because the target classes do not have exactly equal counts.

```
cv = StratifiedKFold(n_splits=5)
cv_results = {}

for model_name, model in MODELS.items():
    scores = cross_val_score(
        model,
        X_train,
        y_train,
        cv=cv,
        scoring="f1_weighted",
    )
    cv_results[model_name] = {
        "mean_f1_weighted": scores.mean(),
        "std_f1_weighted": scores.std(),
        "scores": scores,
    }
    print(
        f"{model_name}: "
        f"mean CV F1 = {scores.mean():.4f}, "
        f"std CV F1 = {scores.std():.4f}"
    )
```

I store the results in cv_results.

I print the model name, the mean CV F1, and the standard deviation.

Output I get:

```
X_train shape: (32028, 14)
X_test shape: (8008, 14)
y_train shape: (32028,)
y_test shape: (8008,)
```

This tells me:

- ♦ I have 32,028 rows for training.
- ♦ I have 8,008 rows for testing.
- ♦ I have 14 selected features.
- ♦ The target files load as one-dimensional label arrays.

```
y_test_drop1 = y_test.drop(00000)
Logistic Regression: mean CV F1 = 0.8739, std CV F1 = 0.0031
Decision Tree: mean CV F1 = 0.9158, std CV F1 = 0.0026
Random Forest: mean CV F1 = 0.9227, std CV F1 = 0.0029
Neural Network (MLP): mean CV F1 = 0.9048, std CV F1 = 0.0049
Logistic Regression: training time = 0.22 seconds
Decision Tree: training time = 0.18 seconds
Random Forest: training time = 5.02 seconds
Neural Network (MLP): training time = 108.12 seconds
```

From these results, I can see that Random Forest gives me the best cross-validation weighted F1 score among the four models.

The ranking by mean CV F1 is:

- 1- Random Forest - 0.9227
- 2- Decision Tree - 0.9158
- 3- Neural Network (MLP) - 0.9048
- 4- Logistic Regression - 0.8739

Step 4 - Fit All Models on the Full Training Set

After cross-validation, I fit every model on the full training set.

I time each model fit using:

```
for model_name, model in MODELS.items():
    start_time = time.time()
    model.fit(X_train, y_train)
    elapsed_time = time.time() - start_time
    training_times[model_name] = elapsed_time
    print(f"{model_name}: training time = {elapsed_time:.2f} seconds")
```

The Neural Network takes the longest time, which makes sense because it trains multiple layers and uses iterative optimization. The tree-based models train much faster in this dataset.

Step 5 - Plot CV Results Comparison

I create a bar chart to compare the mean weighted F1 scores for all four models.

The chart includes:

- 1- One bar for each model.
- 2- Error bars using the CV standard deviation.
- 3- Value labels on top of each bar.
- 4- A colorblind-friendly palette.

```

PLOTS_DIR.mkdir(exist_ok=True)

model_names = list(cv_results.keys())
mean_scores = [cv_results[name]["mean_f1_weighted"] for name in model_names]
std_scores = [cv_results[name]["std_f1_weighted"] for name in model_names]

colorblind_palette = ["#0072B2", "#E69F00", "#009E73", "#CC79A7"]

fig, ax = plt.subplots(figsize=(11, 6))
bars = ax.bar(
    model_names,
    mean_scores,
    yerr=std_scores,
    capsize=6,
    color=colorblind_palette,
    edgecolor="black",
    linewidth=0.8,
)

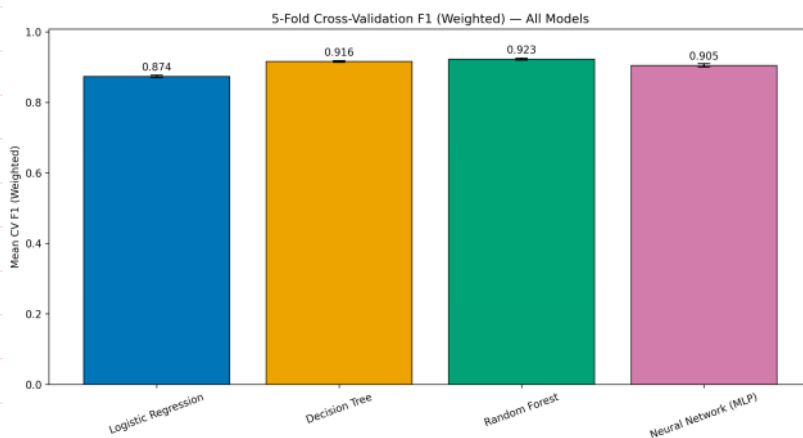
ax.set_title("5-Fold Cross-Validation F1 (Weighted) - All Models")
ax.set_ylabel("Mean CV F1 (Weighted)")
ax.set_ylim(0, max(mean_scores) + max(std_scores) + 0.08)
ax.tick_params(axis="x", rotation=20)

for bar, score in zip(bars, mean_scores):
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 0.01,
        f"{score:.3f}",
        ha="center",
        va="bottom",
        fontsize=10,
    )

fig.tight_layout()
cv_plot_path = PLOTS_DIR / "cv_f1_weighted_all_models.png"
fig.savefig(cv_plot_path, dpi=300, bbox_inches="tight")
plt.close(fig)

print(f"CV results plot saved to {cv_plot_path}")

```



This plot helps me quickly compare the models visually. Random Forest appears as the best model because its bar has the highest mean weighted F1 score.

Step 6 - Feature Importance Using Random Forest

After fitting the Random Forest model, I extract feature importances, and load the feature names:

```

feature_names = [
    line.strip()
    for line in (DATA_DIR / "selected_features.txt").read_text().splitlines()
    if line.strip()
]

random_forest = MODELS["Random Forest"]
feature_importances = random_forest.feature_importances_

```

I combine the feature names and importance values into a dataframe, sort them, and keep the top 10.

The top 10 Random Forest feature importances are:

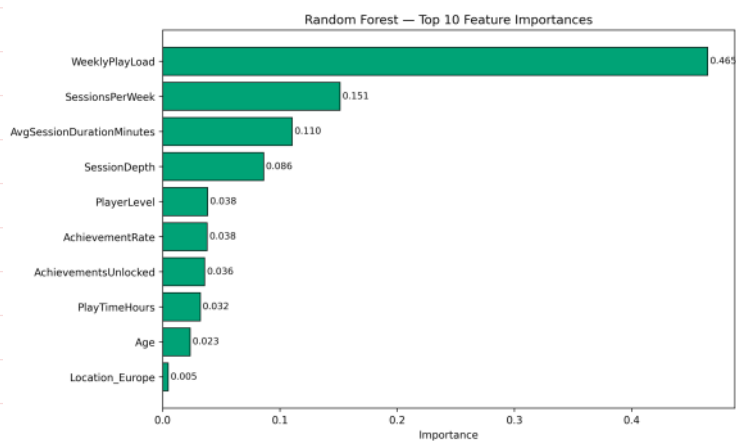
Rank	Feature	Importance
1	WeeklyPlayLoad	0.464758
2	SessionsPerWeek	0.151367
3	AvgSessionDurationMinutes	0.110363
4	SessionDepth	0.086094
5	PlayerLevel	0.038296
6	AchievementRate	0.037983
7	AchievementsUnlocked	0.035878
8	PlayTimeHours	0.032062
9	Age	0.023480
10	Location_Europe	0.004757

The most important feature is: WeeklyPlayLoad

This makes sense because WeeklyPlayLoad combines session frequency and average session duration, so it gives the model a strong signal about how actively a player engages with the platform.

Output I get:

Feature importance plot saved to plots\random_forest_top_10_feature_importances.png
Saved Logistic Regression to models\logistic_regression.pkl



```

feature_names = []
line.strip()
for line in (DATA_DIR / "selected_features.txt").read_text().splitlines():
    if line.strip():
        feature_names.append(line.strip())

random_forest = MODELS["Random Forest"]
feature_importances_ = random_forest.feature_importances_

importance_df = (
    pd.DataFrame(
        {
            "feature": feature_names,
            "importance": feature_importances_,
        }
    )
    .sort_values("importance", ascending=False)
    .head(10)
    .sort_values("importance", ascending=True)
)

fig, ax = plt.subplots(figsize=(10, 6))
ax.barh(
    importance_df["feature"],
    importance_df["importance"],
    color="#0096F3",
    edgecolor="black",
    linewidth=0.8,
)

ax.set_title("Random Forest - Top 10 Feature Importances")
ax.set_xlabel("Importance")

for index, value in enumerate(importance_df["importance"]):
    ax.text(value + 0.002, index, f"{value:.3f}", va="center", fontsize=9)

fig.tight_layout()
feature_importance_plot_path = PLOTS_DIR / "random_forest_top_10_feature_importances.png"
fig.savefig(feature_importance_plot_path, dpi=300, bbox_inches="tight")
plt.close(fig)

print(f"Feature importance plot saved to {feature_importance_plot_path}")

```

Step 7 - Save All Models

I create the models/ directory if it does not already exist:

```

MODELS_DIR.mkdir(exist_ok=True)

def to_snake_case(model_name):
    return re.sub(r"[_]+", "_", re.sub(r"^[a-z0-9]+", "", model_name.lower())).strip("_")

```

I also create a helper function that converts model names into snake_case filenames. This converts names like : Neural Network (MLP) into neural_network_mlp

Then I save each trained model with joblib.dump():

```

for model_name, model in MODELS.items():
    model_path = MODELS_DIR / f"{to_snake_case(model_name)}.joblib"
    joblib.dump(model, model_path)
    print(f"Saved {model_name} to {model_path}")

```

Output I get:

```

Feature importance plot saved to plots/random_forest_top_10_feature_importances.png
Saved Logistic Regression to models\logistic_regression.joblib
Saved Decision Tree to models\decision_tree.joblib
Saved Random Forest to models\random_forest.joblib
Saved Neural Network (MLP) to models\neural_network_mlp.joblib
All 4 models saved to models/

```

The final model files are:

- ♦ models/logistic_regression.joblib
- ♦ models/decision_tree.joblib

- ♦ models/random_forest.joblib
- ♦ models/neural_network_mlp.joblib

Full Pipeline Code Summary

- 1- I import the required scikit-learn models, cross-validation tools, plotting libraries, timing tools, and joblib.
- 2- I load X_train, X_test, y_train, and y_test.
- 3- I define four models inside the MODELS dictionary.
- 4- I run 5-fold stratified cross-validation using f1_weighted.
- 5- I store each model's mean score, standard deviation, and fold scores inside cv_results.
- 6- I fit every model on the full training set.
- 7- I measure and print the training time for each model.
- 8- I create a cross-validation comparison chart.
- 9- I extract Random Forest feature importances.
- 10- I create a top-10 feature-importance chart.
- 11- I create the models/ folder.
- 12- I save all four trained models as .joblib files.

Some Problems

1- The full run takes longer than a short terminal timeout

When I first run the full pipeline, the command reaches the timeout before the script finishes. The reason is the MLP cross-validation step. With 5 folds and max_iter=300, the neural network training takes a long time on this dataset.

To understand the timing better, I test the models separately.

The first three models finish cross-validation much faster:

```
Logistic Regression: training time = 0.22 seconds
Decision Tree: training time = 0.18 seconds
Random Forest: training time = 5.02 seconds
```

The MLP cross-validation takes much longer:

```
Random Forest: training time = 5.02 seconds
Neural Network (MLP): training time = 108.12 seconds
CV results plot saved to plots\cv_f1_weighted_all_models.png
```

After I confirm that the MLP is the slow part, I run the full script with output logs so I can monitor progress.

2- MLP reaches max_iter before convergence

During MLP training, scikit-learn prints convergence warnings:

"ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization hasn't converged yet."

The warning means the optimizer reaches the maximum number of iterations before fully converging.

The model still trains and produces CV scores and a final fitted model file.

3- Plot windows can interrupt terminal execution

The script needs to generate plots, but I want the training pipeline to run fully from the terminal.

To avoid plot windows interrupting the run, I set: `matplotlib.use("Agg")`

Then I save each plot as a PNG file and close the figure:

```
fig.tight_layout()
cv_plot_path = PLOTS_DIR / "cv_f1_weighted_all_models.png"
fig.savefig(cv_plot_path, dpi=300, bbox_inches="tight")
plt.close(fig)
```

This keeps the script clean for command-line execution.

Project Tree

```
github/
|-- data/
|   |-- X_train.csv
|   |-- X_test.csv
|   |-- y_train.csv
|   |-- y_test.csv
|   |-- selected_features.txt
|-- models/
|   |-- logistic_regression.joblib
|   |-- decision_tree.joblib
|   |-- random_forest.joblib
|   |-- neural_network_mlp.joblib
|-- plots/
|   |-- cv_f1_weighted_all_models.png
|   |-- random_forest_top_10_feature_importances.png
|-- model_training_pipeline.py
|-- training_run.log
|-- training_run.err
`-- requirements.txt
```

Final Result

At the end of this task, I have a complete model training pipeline in: `model_training_pipeline.py`

The pipeline trains and compares four models:

- 1- Logistic Regression
- 2- Decision Tree
- 3- Random Forest
- 4- Neural Network (MLP)

The best cross-validation result comes from Random Forest:

Decision Tree: mean CV F1 = 0.9158, std CV F1 = 0.0026

Random Forest: mean CV F1 = 0.9227, std CV F1 = 0.0029

Neural Network (MLP): mean CV F1 = 0.9048, std CV F1 = 0.0049

Random Forest: mean CV F1 = 0.9227, std CV F1 = 0.0029

This means the project now has trained model artifacts that I can load later for testing, evaluation, prediction, or deployment.

Model Evaluation & Comparison

Task Objective

The objective of Task 5 is to evaluate all four saved models on the held-out test set and identify the best model using weighted F1 score.

The required saved models I evaluate are:

- 1- Decision Tree
- 2- Logistic Regression
- 3- Neural Network MLP
- 4- Random Forest

The required test metrics are:

- 1- Accuracy
- 2- Weighted precision
- 3- Weighted recall
- 4- Weighted F1 score
- 5- Weighted one-vs-rest ROC-AUC

The notebook I create for this task is: model_evaluation_pipeline.ipynb

Step 1 - Imports

I import the main libraries for loading models, calculating metrics, and plotting charts:

```
import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from IPython.display import display
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
    auc,
)
from sklearn.preprocessing import label_binarize
```

I use label_binarize : to convert multiclass labels into one-vs-rest format for ROC curves.

I also set the plot theme: sns.set_theme(style='whitegrid')

This keeps the evaluation charts clean and readable.

Step 2 - Load Models and Test Data

I define the project folders:

```
BASE_DIR = Path.cwd()
MODELS_DIR = BASE_DIR / 'models'
DATA_DIR = BASE_DIR / 'data'
RESULTS_DIR = BASE_DIR / 'results'
RESULTS_DIR.mkdir(exist_ok=True)
```

I define the class labels and class names:

```
CLASS_LABELS = [0, 1, 2]
CLASS_NAMES = ['Low', 'Medium', 'High']
```

Then I define the model paths:

```
model_paths = {
    'Decision Tree': MODELS_DIR / 'decision_tree.joblib',
    'Logistic Regression': MODELS_DIR / 'logistic_regression.joblib',
    'Neural Network MLP': MODELS_DIR / 'neural_network_mlp.joblib',
    'Random Forest': MODELS_DIR / 'random_forest.joblib',
}
```

I load every model with `joblib.load()`:

```
models = {name: joblib.load(path) for name, path in model_paths.items()}
```

Then I load the test set:

```
X_test = pd.read_csv(DATA_DIR / 'X_test.csv')
y_test = pd.read_csv(DATA_DIR / 'y_test.csv').iloc[:, 0]
```

Output I get:

```
Loaded 4 models
X_test shape: (8008, 14)
y_test shape: (8008,)
```

This confirms that the notebook sees all four saved models and the correct test data.

Step 3 - Predict with Each Model

For each model, I generate:

- ♦ `y_pred`
- ♦ `y_proba`

The prediction loop:

```

predictions = {}

for model_name, model in models.items():
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)

    predictions[model_name] = {
        'model': model,
        'y_pred': y_pred,
        'y_proba': y_proba,
    }

print(f'{model_name}: generated predictions and probabilities')

```

I need both outputs because:

Output	Purpose
y_pred	Needed for accuracy, precision, recall, F1, classification report, and confusion matrix
y_proba	Needed for ROC-AUC and ROC curve plotting

Output I get:

```

Decision Tree: generated predictions and probabilities
Logistic Regression: generated predictions and probabilities
Neural Network MLP: generated predictions and probabilities
Random Forest: generated predictions and probabilities

```

I also verify that every model supports " predict_proba() " and returns three probability columns, one for each class.

Step 4 - Metrics Table

For every model, I calculate the required metrics:

```

for model_name, outputs in predictions.items():
    y_pred = outputs['y_pred']
    y_proba = outputs['y_proba']

    metrics_rows.append({
        'Model': model_name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision (Weighted)': precision_score(y_test, y_pred, average='weighted', zero_division=0),
        'Recall (Weighted)': recall_score(y_test, y_pred, average='weighted', zero_division=0),
        'F1-Score (Weighted)': f1_score(y_test, y_pred, average='weighted', zero_division=0),
        'ROC-AUC (OvR Weighted)': roc_auc_score(
            y_test,
            y_proba,
            labels=CLASS_LABELS,
            multi_class='ovr',
            average='weighted',
        ),
    })

```

I use average = 'weighted'

because this is a multiclass problem and the class support counts are not identical. Weighted metrics give each class influence based on how many examples it has.

I use multi_class='ovr'

for ROC-AUC because the model is predicting three classes.
One-vs-rest ROC-AUC treats each class as positive against the other two classes.

Metrics table I get:

Model Evaluation Metrics					
	Accuracy	Precision (Weighted)	Recall (Weighted)	F1-Score (Weighted)	ROC-AUC (OvR Weighted)
Model					
Decision Tree	0.9170	0.9170	0.9170	0.9167	0.9299
Logistic Regression	0.8764	0.8810	0.8764	0.8753	0.9339
Neural Network MLP	0.9175	0.9175	0.9175	0.9172	0.9433
Random Forest	0.9245	0.9247	0.9245	0.9242	0.9467

I also display the styled table:

```
print('Model Evaluation Metrics')
display(metrics_df.round(4))
display(metrics_df.style.highlight_max(axis=0).format('{:.4f}'))
```

This highlights the highest value in each metric column.
The best model in every metric column is: Random Forest

Step 5 - Classification Reports

For each model, I print a full classification report:

```
for model_name, outputs in predictions.items():
    print('=' * 80)
    print(f'Classification Report ? {model_name}')
    print('=' * 80)
    print(classification_report(
        y_test,
        outputs['y_pred'],
        labels=CLASS_LABELS,
        target_names=CLASS_NAMES,
        zero_division=0,
    ))
```

Decision Tree Classification Report

```
Classification Report ? Decision Tree
=====
=====
              precision    recall  f1-score   support

   Low           0.91         0.89         0.90         2065
   Medium        0.92         0.95         0.93         3876
   High          0.93         0.88         0.90         2067

 accuracy              0.92         8008
 macro avg           0.92         0.91         0.91         8008
 weighted avg        0.92         0.92         0.92         8008
```

The Decision Tree performs strongly overall, with the best recall on the "Medium" class.

Logistic Regression Classification Report

Classification Report ? Logistic Regression

=====

=====

	precision	recall	f1-score	support
Low	0.90	0.78	0.83	2065
Medium	0.84	0.95	0.89	3876
High	0.94	0.83	0.88	2067
accuracy			0.88	8008
macro avg	0.89	0.85	0.87	8008
weighted avg	0.88	0.88	0.88	8008

Logistic Regression gives the lowest weighted F1 score among the four models, but it still has strong precision for the "High" class.

Neural Network MLP Classification Report

Classification Report ? Neural Network MLP

=====

=====

	precision	recall	f1-score	support
Low	0.91	0.89	0.90	2065
Medium	0.92	0.95	0.93	3876
High	0.92	0.89	0.91	2067
accuracy			0.92	8008
macro avg	0.92	0.91	0.91	8008
weighted avg	0.92	0.92	0.92	8008

The Neural Network MLP slightly beats the Decision Tree on weighted F1, but it still remains below Random Forest.

Random Forest Classification Report

Classification Report ? Random Forest

=====

=====

	precision	recall	f1-score	support
Low	0.92	0.90	0.91	2065
Medium	0.92	0.96	0.94	3876
High	0.94	0.89	0.91	2067
accuracy			0.92	8008
macro avg	0.93	0.91	0.92	8008
weighted avg	0.92	0.92	0.92	8008

Random Forest gives the strongest weighted F1 score and the highest overall accuracy.

Step 6 - Confusion Matrices

I plot a 2x2 grid of confusion matrices using Seaborn heatmaps.

```
for ax, (model_name, outputs) in zip(axes, predictions.items()):
    cm = confusion_matrix(y_test, outputs['y_pred'], labels=CLASS_LABELS)

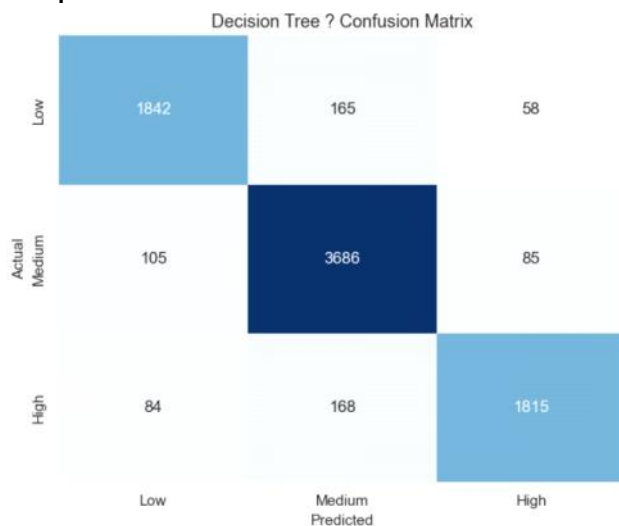
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        xticklabels=CLASS_NAMES,
        yticklabels=CLASS_NAMES,
        cbar=False,
        ax=ax,
    )
    ax.set_title(f'{model_name} ? Confusion Matrix')
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
```

Each plot includes:

- 1- Title: {Model Name} — Confusion Matrix
- 2- X-axis label: Predicted
- 3- Y-axis label: Actual
- 4- Class labels: Low, Medium, High

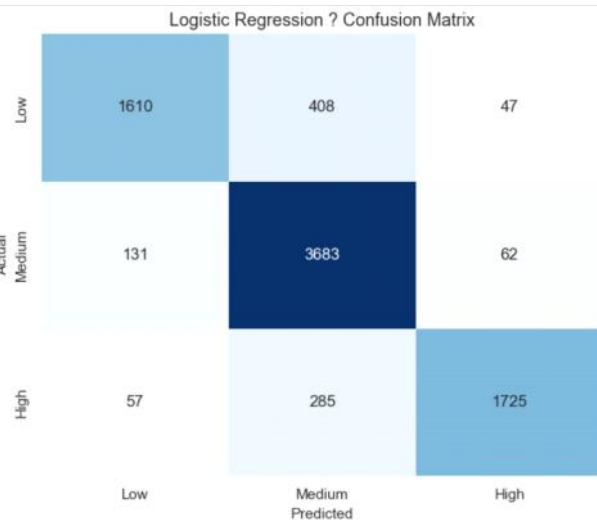
Decision Tree Confusion Matrix

Output values:



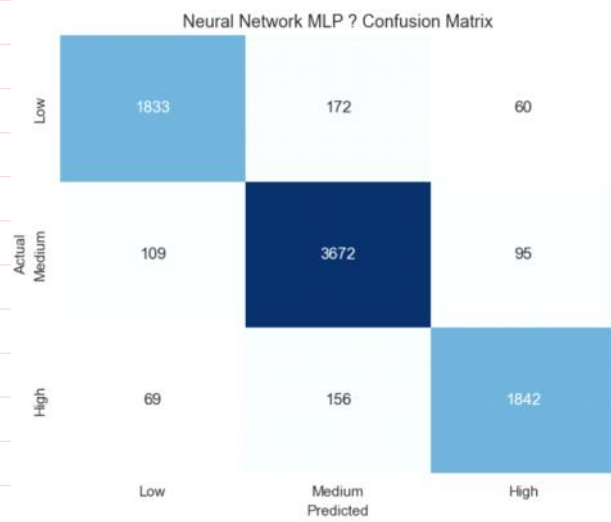
Logistic Regression Confusion Matrix

Output values:



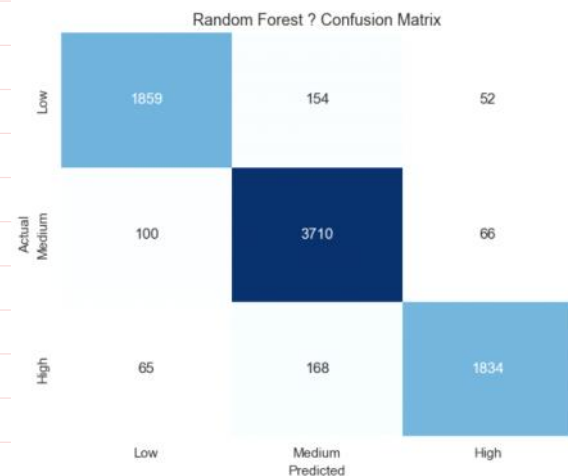
Neural Network MLP Confusion Matrix

Output values:



Random Forest Confusion Matrix

Output values:



The Random Forest confusion matrix shows the strongest diagonal counts overall:

- ♦ Low correctly predicted: 1859
- ♦ Medium correctly predicted: 3710
- ♦ High correctly predicted: 1834

This means it makes the largest number of correct predictions among the four models.

Step 7 - ROC Curves for the Best Model

In this task, I plot ROC curves only for the best model by weighted F1.

I identify the best model with:

```
best_model_name = metrics_df['F1-Score (Weighted)'].idxmax()
best_outputs = predictions[best_model_name]
```

To calculate multiclass ROC curves, I first binarize the test labels:

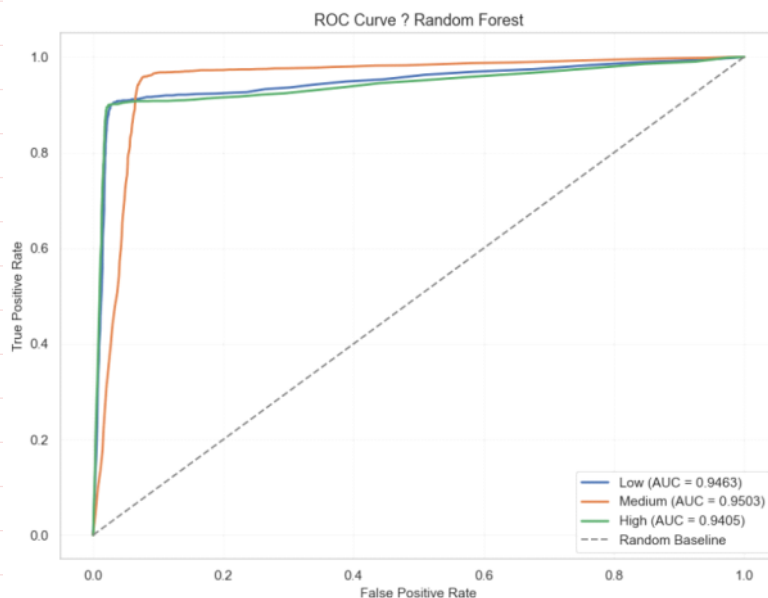
```
best_outputs = predictions[best_model_name]
y_test_binarized = label_binarize(y_test, classes=CLASS_LABELS)
y_proba = best_outputs['y_proba']
```

Then I calculate one ROC curve for each class:

```
for class_index, class_name in enumerate(CLASS_NAMES):
    fpr, tpr, _ = roc_curve(y_test_binarized[:, class_index], y_proba[:, class_index])
    class_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, linewidth=2, label=f'{class_name} (AUC = {class_auc:.4f})')
```

The ROC plot includes:

- 1- One line for Low.
- 2- One line for Medium.
- 3- One line for High.
- 4- A dashed random baseline.
- 5- A legend with AUC values.
- 6- Title: ROC Curve — Random Forest.
- 7- X-axis: False Positive Rate.
- 8- Y-axis: True Positive Rate.



Per-class Random Forest ROC-AUC values:

- ♦ Low: 0.9463
- ♦ Medium: 0.9503
- ♦ High: 0.9405

The highest per-class ROC-AUC is for: Medium

This means the Random Forest separates the Medium engagement class especially well from the other two classes.

Step 8 - Winner Summary

I created a markdown-style winner summary in the notebook:

```
best_metrics = metrics_df.loc[best_model_name]

print('''
## Winner Summary
| Field | Value |
|---|---|
| Best Model | {model_name} |
| Test F1-Score (Weighted) | {f1:.4f} |
| Test Accuracy | {accuracy:.4f} |
| Test ROC-AUC (OvR Weighted) | {roc_auc:.4f} |

Business interpretation: For BabiPoly, this means the {model_name} model gives
'''.format(
    model_name=best_model_name,
    f1=best_metrics['F1-Score (Weighted)'],
    accuracy=best_metrics['Accuracy'],
    roc_auc=best_metrics['ROC-AUC (OvR Weighted)'],
))
```

```
## Winner Summary
| Field | Value |
|---|---|
| Best Model | Random Forest |
| Test F1-Score (Weighted) | 0.9242 |
| Test Accuracy | 0.9245 |
| Test ROC-AUC (OvR Weighted) | 0.9467 |
```

Business interpretation: For BabiPoly, this means the Random Forest model gives the most balanced held-out performance for predicting player engagement levels, making it the strongest candidate for identifying Low, Medium, and High engagement users in production-style decision support.

This summary is useful because it gives the final answer in one compact place after all

metrics and plots

Step 9 - Save Results

I create the results/ folder if needed: `RESULTS_DIR.mkdir(exist_ok=True)`

Then I save the metrics table:

```
results_path = RESULTS_DIR / 'evaluation_results.csv'
metrics_df.to_csv(results_path)

print(f'Evaluation complete. Best model: {best_model_name} with F1={best_metrics["F1-Score (Weighted)"]:.4f}')
print(f'Results saved to: {results_path}')
```

Some Problems

1- I need y_test as a one-dimensional series

The file `data/y_test.csv` loads as a one-column dataframe by default.

Fix:

I select the first column:

```
x_test = pd.read_csv(DATA_DIR / 'x_test.csv')
y_test = pd.read_csv(DATA_DIR / 'y_test.csv').iloc[:, 0]
```

This gives the metrics functions a clean one-dimensional target series.

2- Multiclass ROC-AUC needs probability outputs

For ROC-AUC, class labels alone are not enough. I need probability scores for each class.

Fix:

For each model, I call:

```
for model_name, model in models.items():
    y_pred = model.predict(x_test)
    y_proba = model.predict_proba(x_test)
```

All four saved classifiers provide `predict_proba()` and return probability arrays with this shape: `(8008, 3)`

This matches the three engagement classes.

3- ROC curves need one-vs-rest labels

The test labels are multiclass labels: `0, 1, 2`

To draw one ROC curve per class, I convert them into binary columns:

```
best_outputs = predictions[best_model_name]
y_test_binarized = label_binarize(y_test, classes=CLASS_LABELS)
y_proba = best_outputs['y_proba']
```

Then each class can be compared against the other two classes.

Business Interpretation for BabiPoly

For BabiPoly, the Random Forest model is the best current choice because it gives the most balanced prediction performance across Low, Medium, and High player engagement levels.

This means BabiPoly can use this model to better identify:

- 1- Low-engagement players who may need retention actions.
- 2- Medium-engagement players who may respond well to nudges or personalized offers.
- 3- High-engagement players who may be strong candidates for loyalty programs, tournaments, or premium features.

The strongest result is useful because engagement prediction can help the platform make smarter decisions about player retention, personalization, and product planning.

RF Hyperparameter Tuning

Task Objective

The objective of Task 6 is to run a two-stage hyperparameter tuning process for the Random Forest model.

The tuning workflow is:

- 1- Load the train/test files from data/.
- 2- Load the baseline Random Forest model from models/random_forest.joblib.
- 3- Evaluate the baseline model on the held-out test set.
- 4- Run a broad RandomizedSearchCV.
- 5- Use the Stage 1 best parameters to build a narrower GridSearchCV.
- 6- Retrain the final tuned Random Forest on the full training set.
- 7- Evaluate the tuned model on the held-out test set.
- 8- Print a baseline-vs-tuned comparison table.
- 9- Plot a learning curve for the tuned model.
- 10- Save the tuned model and best parameters.

The main tuning metric is: ' f1_weighted '

I use weighted F1 because this is a multiclass classification problem and the class distribution is not perfectly equal. Weighted F1 balances precision and recall while also respecting class support.

Step 1 - Baseline Score

I start by importing the libraries:

```
# %%
from pathlib import Path
import json
import time

import joblib
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV, learning_curve
```

I use:

- ♦ ' RandomForestClassifier ' : Build the tuned model

- ♦ ' RandomizedSearchCV ' : Run the broad Stage 1 search
- ♦ ' GridSearchCV ' : Run the narrow Stage 2 search
- ♦ ' learning_curve ' : Create the tuned model learning curve

I define the project folders, create the output folders if they do not already exist, then I load the train/test files:

```
# %%
DATA_DIR = Path("data")
MODELS_DIR = Path("models")
RESULTS_DIR = Path("results")
PLOTS_DIR = Path("plots")

RESULTS_DIR.mkdir(exist_ok=True)
PLOTS_DIR.mkdir(exist_ok=True)

X_train = pd.read_csv(DATA_DIR / "X_train.csv")
X_test = pd.read_csv(DATA_DIR / "X_test.csv")
y_train = pd.read_csv(DATA_DIR / "y_train.csv").squeeze("columns")
y_test = pd.read_csv(DATA_DIR / "y_test.csv").squeeze("columns")
```

I use .squeeze("columns") for y_train and y_test so the target loads as a one-dimensional series.

Next I load the baseline model, then I evaluate it on the test set:

```
baseline_rf = joblib.load(MODELS_DIR / "random_forest.joblib")
baseline_pred = baseline_rf.predict(X_test)
baseline_proba = baseline_rf.predict_proba(X_test)

baseline_f1 = f1_score(y_test, baseline_pred, average="weighted", zero_division=0)
baseline_accuracy = accuracy_score(y_test, baseline_pred)
baseline_roc_auc = roc_auc_score(
    y_test,
    baseline_proba,
    multi_class="ovr",
    average="weighted",
)
```

The output:

```
STEP 1 - Baseline Random Forest test results
Baseline weighted F1: 0.9242
Baseline accuracy:    0.9245
Baseline ROC-AUC:     0.9467
```

These baseline values match the Random Forest results from Task 5.

Step 2 - Stage1: RandomizedSearchCV Broad Search

The first tuning stage uses a broad random search.

I define the requested parameter distributions:

```
# %%
param_distributions = {
    "n_estimators": [100, 200, 300, 500],
    "max_depth": [None, 10, 20, 30, 40],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2", None],
    "bootstrap": [True, False],
}
```

What each parameter controls:

Parameter	Meaning
n_estimators	Number of trees in the forest
max_depth	Maximum depth allowed for each tree
min_samples_split	Minimum samples needed to split an internal node
min_samples_leaf	Minimum samples required in a leaf node
max_features	Number of features considered at each split
bootstrap	Whether each tree trains on bootstrap samples

I create the Stage 1 model:

```
stage1_model = RandomForestClassifier(random_state=42, n_jobs=1)
```

I use n_jobs=1 inside the Random Forest because RandomizedSearchCV already uses parallel jobs with.

This helps avoid nested parallel processing that can slow down the machine

The Stage 1 search is:

```
random_search = RandomizedSearchCV(
    estimator=stage1_model,
    param_distributions=param_distributions,
    cv=5,
    n_iter=30,
    scoring="f1_weighted",
    n_jobs=-1,
    random_state=42,
    verbose=1,
)
```

This means:

1- I test 30 random hyperparameter combinations

- 2- Each combination uses 5-fold cross-validation
- 3- The scoring metric is weighted F1
- 4- The search prints progress because verbose=1.

Total Stage 1 fits:

```
STEP 2 - Starting RandomizedSearchCV broad search
Fitting 5 folds for each of 30 candidates, totalling 150 fits
```

After fitting:

```
print("\nSTEP 2 - Starting RandomizedSearchCV broad search")
stage1_start = time.time()
random_search.fit(X_train, y_train)
stage1_elapsed = time.time() - stage1_start

stage1_best_params = random_search.best_params_
stage1_best_cv_f1 = random_search.best_score_

print("\nSTEP 2 - RandomizedSearchCV results")
print(f"Stage 1 runtime: {stage1_elapsed / 60:.2f} minutes")
print("Best params:")
print(stage1_best_params)
print(f"Best CV F1: {stage1_best_cv_f1:.4f}")
```

Output:

```
STEP 2 - Starting RandomizedSearchCV broad search
Fitting 5 folds for each of 30 candidates, totalling 150 fits

STEP 2 - RandomizedSearchCV results
Stage 1 runtime: 39.52 minutes
Best params:
{'n_estimators': 500, 'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': None, 'max_depth': 30, 'bootstrap': True}
Best CV F1: 0.9270
```

The best broad-search configuration is:

Parameter	Stage 1 Best Value
n_estimators	500
max_depth	30
min_samples_split	2
min_samples_leaf	4
max_features	None
bootstrap	True

The Stage 1 best weighted CV F1 is: 0.9270

This is already slightly higher than the baseline Task 5 test F1 of 0.9242, so the random search finds a promising tuning direction.

Step 3 - Stage 2: GridSearchCV Narrow Search

The second tuning stage uses the best parameters from Stage 1 to build a narrower grid.

I create this helper function:

```
# %%
def values_around_best(best_value, ordered_values):
    """Return the best value plus its immediate neighbors in an ordered grid."""
    best_index = ordered_values.index(best_value)
    start_index = max(best_index - 1, 0)
    end_index = min(best_index + 2, len(ordered_values))
    return ordered_values[start_index:end_index]
```

This function selects:

- 1- The best value from Stage 1.
- 2- The value one step before it, if one exists.
- 3- The value one step after it, if one exists.

Then I build the narrow grid:

```
narrow_param_grid = {
    "n_estimators": values_around_best(
        stage1_best_params["n_estimators"],
        param_distributions["n_estimators"],
    ),
    "max_depth": values_around_best(
        stage1_best_params["max_depth"],
        param_distributions["max_depth"],
    ),
    "min_samples_split": values_around_best(
        stage1_best_params["min_samples_split"],
        param_distributions["min_samples_split"],
    ),
    "min_samples_leaf": values_around_best(
        stage1_best_params["min_samples_leaf"],
        param_distributions["min_samples_leaf"],
    ),
    "max_features": values_around_best(
        stage1_best_params["max_features"],
        param_distributions["max_features"],
    ),
    # Bootstrap is boolean, so the narrow grid keeps the Stage 1 winner.
    "bootstrap": [stage1_best_params["bootstrap"]],
}
```

I keep bootstrap fixed to the Stage 1 winner because it is a boolean value, so it does not have a real plus-or-minus-one neighborhood.

I also print the number of narrow-grid candidates:

```
print("\nSTEP 3 - Narrow parameter grid")
print(narrow_param_grid)
grid_candidate_count = int(np.prod([len(values) for values in narrow_param_grid.values()]))
print(f"Narrow grid candidates: {grid_candidate_count}")
print(f"Narrow grid total CV fits: {grid_candidate_count * 5}")
```


This helps me understand how long Stage 2 may take.

The Stage 2 search is:

```
grid_search = GridSearchCV(  
    estimator=stage2_model,  
    param_grid=narrow_param_grid,  
    cv=5,  
    scoring="f1_weighted",  
    n_jobs=-1,  
    verbose=1,  
)
```

After fitting, I print:

```
final_best_params = grid_search.best_params_  
final_best_cv_f1 = grid_search.best_score_  
  
print("\nSTEP 3 - GridSearchCV results")  
print(f"Stage 2 runtime: {stage2_elapsed / 60:.2f} minutes")  
print("Final best params:")  
print(final_best_params)  
print(f"Final best CV F1: {final_best_cv_f1:.4f}")
```

These final parameters become the settings for the tuned Random Forest model.

Output I get:

```
STEP 3 - Narrow parameter grid  
{'n_estimators': [300, 500], 'max_depth': [20, 30, 40], 'min_samples_split': [2, 5], 'min_samples_leaf': [2, 4], 'max_features': ['log2', None], 'bootstrap': [True]}  
Narrow grid candidates: 48  
Narrow grid total CV fits: 240
```

```
STEP 3 - Starting GridSearchCV narrow search  
Fitting 5 folds for each of 48 candidates, totalling 240 fits  
  
STEP 3 - GridSearchCV results  
Stage 2 runtime: 84.45 minutes  
Final best params:  
{'bootstrap': True, 'max_depth': 20, 'max_features': None, 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 300}  
Final best CV F1: 0.9270
```

The final best parameters are:

Parameter	Final Best Value
bootstrap	True
max_depth	20
max_features	None
min_samples_leaf	2
min_samples_split	5
n_estimators	300

The final Stage 2 weighted CV F1 is: 0.9270

The final CV F1 stays very close to the Stage 1 CV F1, but the selected model becomes a little simpler than the Stage 1 winner because it uses 300 trees instead of 500.

Step 4 - Retrain with Best Params

After the two-stage search, I retrain a new Random Forest using the final best parameters:

```
# %%  
tuned_rf = RandomForestClassifier(  
    **final_best_params,  
    random_state=42,  
    n_jobs=-1,  
)
```

Then I fit it on the full training set:

```
print("\nSTEP 4 - Retraining tuned Random Forest on full X_train")  
retrain_start = time.time()  
tuned_rf.fit(X_train, y_train)  
retrain_elapsed = time.time() - retrain_start
```

This is important because the search uses cross-validation folds, but the final model should train on all available training rows before test evaluation.

Then I evaluate the tuned model on the held-out test set:

```
tuned_pred = tuned_rf.predict(X_test)  
tuned_proba = tuned_rf.predict_proba(X_test)  
  
tuned_f1 = f1_score(y_test, tuned_pred, average="weighted", zero_division=0)  
tuned_accuracy = accuracy_score(y_test, tuned_pred)  
tuned_roc_auc = roc_auc_score(  
    y_test,  
    tuned_proba,  
    multi_class="ovr",  
    average="weighted",  
)
```

The script prints:

```
STEP 4 - Retraining tuned Random Forest on full X_train  
  
STEP 4 - Tuned Random Forest test results  
Retraining time: 35.76 seconds  
Tuned weighted F1: 0.9276  
Tuned accuracy:    0.9278  
Tuned ROC-AUC:    0.9466
```

These are the final tuned test-set results.

- ♦ The tuned Random Forest improves the held-out weighted F1 from: 0.9242 -> 0.9276
- ♦ It also improves test accuracy from: 0.9245 -> 0.9278
- ♦ The ROC-AUC is almost unchanged, with a very small decrease: 0.9467 -> 0.9466

Step 5 - Improvement Summary

I create a comparison dataframe with three rows:

- 1- Baseline RF
- 2- Tuned RF
- 3- Improvement (%)

The dataframe code is:

```
# %%
comparison_df = pd.DataFrame(
    [
        {
            "Model": "Baseline RF",
            "Test F1": baseline_f1,
            "Accuracy": baseline_accuracy,
            "ROC-AUC": baseline_roc_auc,
        },
        {
            "Model": "Tuned RF",
            "Test F1": tuned_f1,
            "Accuracy": tuned_accuracy,
            "ROC-AUC": tuned_roc_auc,
        },
        {
            "Model": "Improvement (%)",
            "Test F1": ((tuned_f1 - baseline_f1) / baseline_f1) * 100,
            "Accuracy": ((tuned_accuracy - baseline_accuracy) / baseline_accuracy)
            * 100,
            "ROC-AUC": ((tuned_roc_auc - baseline_roc_auc) / baseline_roc_auc)
            * 100,
        },
    ],
)
```

The improvement formula is:

$$((\text{tuned score} - \text{baseline score}) / \text{baseline score}) * 100$$

This tells me whether tuning actually improves the model compared with the original saved Random Forest.

Output:

```
STEP 5 - Baseline vs Tuned Random Forest
      Model  Test F1  Accuracy  ROC-AUC
Baseline RF   0.9242    0.9245    0.9467
Tuned RF      0.9276    0.9278    0.9466
Improvement (%) 0.3657    0.3647   -0.0124
```

Final comparison:

Model	Test F1	Accuracy	ROC-AUC
Baseline RF	0.9242	0.9245	0.9467
Tuned RF	0.9276	0.9278	0.9466
Improvement (%)	0.3657	0.3647	-0.0124

This means the tuned model gives a small improvement in weighted F1 and accuracy, while ROC-AUC stays essentially the same.

Step 6 - Learning Curve

After training the tuned model, I calculate a learning curve.

The purpose of the learning curve is to compare:

- 1- Training F1-weighted score.
- 2- Validation F1-weighted score.
- 3- How both scores change as the training set size increases.

I create a separate Random Forest for the learning-curve calculation.

```
learning_curve_rf = RandomForestClassifier(  
    **final_best_params,  
    random_state=42,  
    n_jobs=1,  
)
```

I use `n_jobs=1` inside this model because `learning_curve()` itself runs in parallel:

The learning-curve call is:

```
train_sizes, train_scores, validation_scores = learning_curve(  
    estimator=learning_curve_rf,  
    X=X_train,  
    y=y_train,  
    cv=5,  
    scoring="f1_weighted",  
    n_jobs=-1,  
    train_sizes=np.linspace(0.1, 1.0, 5),  
    shuffle=True,  
    random_state=42,  
)
```

The training sizes are:

- ♦ 10%

- ♦ 32.5%
- ♦ 55%
- ♦ 77.5%
- ♦ 100%

```
train_sizes, train_scores, validation_scores = learning_curve(
    estimator=learning_curve_rf,
    X=X_train,
    y=y_train,
    cv=5,
    scoring="f1_weighted",
    n_jobs=-1,
    train_sizes=np.linspace(0.1, 1.0, 5),
    shuffle=True,
    random_state=42,
)
```

For each training size, the function calculates:

- ♦ Training scores across CV folds.
- ♦ Validation scores across CV folds.

Then I calculate the mean and standard deviation:

```
train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
validation_mean = validation_scores.mean(axis=1)
validation_std = validation_scores.std(axis=1)
```

I plot the training score and validation score, and I also add shaded confidence bands:

```
ax.plot(
    train_sizes,
    train_mean,
    marker="o",
    linewidth=2,
    label="Training F1-weighted",
)
ax.fill_between(
    train_sizes,
    train_mean - train_std,
    train_mean + train_std,
    alpha=0.2,
)

ax.plot(
    train_sizes,
    validation_mean,
    marker="s",
    linewidth=2,
    label="Validation F1-weighted",
)
ax.fill_between(
    train_sizes,
    validation_mean - validation_std,
    validation_mean + validation_std,
    alpha=0.2,
)
```

The plot labels are:

```

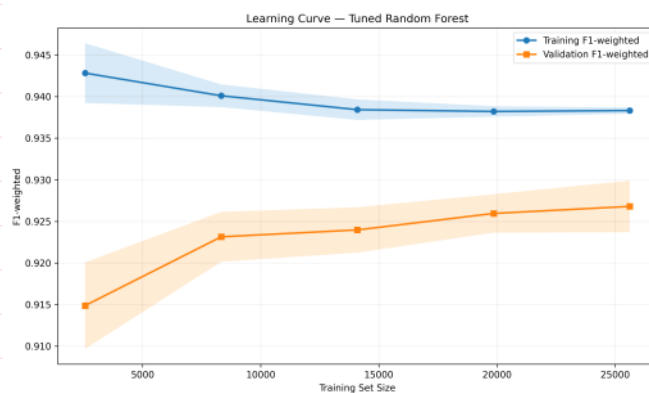
ax.set_title("Learning Curve - Tuned Random Forest")
ax.set_xlabel("Training Set Size")
ax.set_ylabel("F1-weighted")
ax.legend(loc="best")
ax.grid(alpha=0.3)

```

I save the learning curve to: plots/learning_curve_tuned_random_forest.png

This plot helps me understand whether the tuned Random Forest is overfitting, underfitting, or still benefiting from more training data.

STEP 6 - Calculating learning curve for tuned Random Forest
Learning curve saved to plots\learning_curve_tuned_random_forest.png



Step 7 - Save Tuned Model and Best Params

After the final tuned model is trained, I save it to: models/random_forest_tuned.joblib

```

# %%
tuned_model_path = MODELS_DIR / "random_forest_tuned.joblib"
best_params_path = RESULTS_DIR / "best_params.json"

```

I also save the final best hyperparameters to: results/best_params.json

```
# %%
tuned_model_path = MODELS_DIR / "random_forest_tuned.joblib"
best_params_path = RESULTS_DIR / "best_params.json"

joblib.dump(tuned_rf, tuned_model_path)

with best_params_path.open("w", encoding="utf-8") as file:
    json.dump(final_best_params, file, indent=4)
```

Final print statement:

```
print("\nSTEP 7 - Saved tuned artifacts")
print(f"Tuned model path: {tuned_model_path}")
print(f"Best params path: {best_params_path}")
print(f"Tuned model saved. Final F1: {tuned_f1:.4f}")
```

This gives me a saved tuned model and a readable JSON file containing the final selected hyperparameters.

```
STEP 7 - Saved tuned artifacts
Tuned model path: models\random_forest_tuned.joblib
Best params path: results\best_params.json
Tuned model saved. Final F1: 0.9276
```

The final best_params.json file contains:

```
{
  "bootstrap": true,
  "max_depth": 20,
  "max_features": null,
  "min_samples_leaf": 2,
  "min_samples_split": 5,
  "n_estimators": 300
}
```

Final Result

At this point, Task 6 is complete.

I have a complete Random Forest tuning pipeline in: random_forest_tuning_pipeline.py

The script performs:

- 1- Baseline Random Forest test evaluation.
- 2- Broad RandomizedSearchCV.
- 3- Narrow GridSearchCV.
- 4- Final tuned model retraining.
- 5- Baseline-vs-tuned comparison.

- 6- Tuned model learning curve.
- 7- Tuned model and best-parameter saving.

The verified baseline before tuning is:

Model	Test F1	Accuracy	ROC-AUC
Baseline RF	0.9242	0.9245	0.9467

The final tuned model result is:

Model	Test F1	Accuracy	ROC-AUC
Tuned RF	0.9276	0.9278	0.9466

Final improvement:

Metric	Improvement
Test F1	0.3657%
Accuracy	0.3647%
ROC-AUC	-0.0124%

Business Interpretation

For BabiPoly, this tuning step checks whether the already strong Random Forest model can be improved before it is treated as the final engagement prediction model.

The tuned model improves weighted F1 and accuracy by about 0.36%, which means BabiPoly gets a slightly stronger engagement classifier for separating Low, Medium, and High engagement players.

The ROC-AUC decreases by only 0.0124%, so the model's probability-ranking ability stays essentially stable. The main practical gain from tuning is a small improvement in final class prediction quality, not a major change in ROC-AUC.



FastAPI REST API

Task Objective

The objective of Task 7 is to build a production-ready FastAPI REST API for serving player engagement predictions.

The API needs to:

- 1- Load the tuned Random Forest model once at startup.
- 2- Load the selected feature names once at startup.
- 3- Accept a player input payload using a Pydantic schema.
- 4- Recreate the Task 3 engineered features.
- 5- Encode categorical fields into training-compatible one-hot columns.
- 6- Align the request feature row to the selected feature list.
- 7- Return the predicted engagement class.
- 8- Return the confidence score for the predicted class.
- 9- Return all probabilities for Low, Medium, and High.
- 10- Provide health and root endpoints for quick testing.
- 11- Enable CORS so a future Streamlit frontend can call the API.

Created API File

In this task, I create four new files inside the api/ folder. These files work together as the backend service layer for the trained model.

api/schemas.py

The schemas.py file contains the Pydantic models for API input and output.

I create the PlayerInput model with these fields:

- ♦ SessionsPerWeek
- ♦ AvgSessionDurationMinutes
- ♦ PlayerLevel
- ♦ AchievementsUnlocked
- ♦ InGamePurchases
- ♦ Gender
- ♦ Location
- ♦ GameGenre

This file controls the structure of the JSON body that /predict accepts. It also adds validation

and example values so Swagger UI can display a clear request example.

Important validation rules in PlayerInput:

- 1- SessionsPerWeek must be an integer greater than or equal to 0.
- 2- AvgSessionDurationMinutes must be a float greater than or equal to 0.
- 3- PlayerLevel must be an integer greater than or equal to 0.
- 4- AchievementsUnlocked must be an integer greater than or equal to 0.
- 5- InGamePurchases must be either 0 or 1.
- 6- Gender, Location, and GameGenre must be non-empty strings.

I also create the PredictionOutput model with:

- ♦ predicted_class
- ♦ confidence
- ♦ all_probabilities

This keeps the API response consistent.

The predicted class can only be:

- ♦ Low
- ♦ Medium
- ♦ High

The confidence is the probability of the predicted class, and all_probabilities returns the probability for every engagement level.

```
from typing import Dict, Literal
from pydantic import BaseModel, Field

class PlayerInput(BaseModel):
    """Input payload for a single player engagement prediction."""
    SessionsPerWeek: int = Field(
        ...,
        ge=0,
        json_schema_extra={"example": 5},
    )
    AvgSessionDurationMinutes: float = Field(
        ...,
        ge=0,
        json_schema_extra={"example": 45.0},
    )
    PlayerLevel: int = Field(
        ...,
        ge=0,
        json_schema_extra={"example": 12},
    )
    AchievementsUnlocked: int = Field(
        ...,
        ge=0,
        json_schema_extra={"example": 8},
    )
    InGamePurchases: int = Field(
        ...,
        ge=0,
        le=1,
        json_schema_extra={"example": 1},
    )
```

```

Gender: str = Field(
    ...,
    min_length=1,
    json_schema_extra={"example": "Male"},
)
Location: str = Field(
    ...,
    min_length=1,
    json_schema_extra={"example": "Nigeria"},
)
GameGenre: str = Field(
    ...,
    min_length=1,
    json_schema_extra={"example": "Strategy"},
)

class Config:
    json_schema_extra = {
        "example": {
            "SessionsPerWeek": 5,
            "AvgSessionDurationMinutes": 45.0,
            "PlayerLevel": 12,
            "AchievementsUnlocked": 8,
            "InGamePurchases": 1,
            "Gender": "Male",
            "Location": "Nigeria",
            "GameGenre": "Strategy",
        }
    }

```

```

class PredictionOutput(BaseModel):
    """Prediction response returned by the engagement model."""
    predicted_class: Literal["Low", "Medium", "High"] = Field(
        ...,
        json_schema_extra={"example": "Medium"},
    )
    confidence: float = Field(
        ...,
        ge=0,
        le=1,
        json_schema_extra={"example": 0.8734},
    )
    all_probabilities: Dict[str, float] = Field(
        ...,
        json_schema_extra={
            "example": {
                "Low": 0.0312,
                "Medium": 0.8734,
                "High": 0.0954,
            }
        },
    )

```

api/model.py

The model.py file contains the model-loading and prediction logic.

This is the main backend logic file. It loads the saved tuned Random Forest model from: `models/random_forest_tuned.joblib`

It also loads the selected feature list from: `data/selected_features.txt`

```

BASE_DIR = Path(__file__).resolve().parents[1]
MODEL_PATH = BASE_DIR / "models" / "random_forest_tuned.joblib"
SELECTED_FEATURES_PATH = BASE_DIR / "data" / "selected_features.txt"
TRAINING_DATA_PATH = BASE_DIR / "data" / "cleaned_gaming.csv"

CATEGORICAL_COLUMNS = ("Gender", "Location", "GameGenre")
CATEGORICAL_PREFIXES = tuple(f"{column}_" for column in CATEGORICAL_COLUMNS)
CLASS_LABELS = {0: "Low", 1: "Medium", 2: "High"}
PROBABILITY_LABELS = ("Low", "Medium", "High")

```

Both artifacts load at module startup, which means they load once when the API starts instead of loading again for every request.

The file includes helper functions for each preprocessing step:

- ◆ `_read_selected_features()`
- ◆ `_add_engineered_features()`
- ◆ `_fit_task3_scaler()`
- ◆ `_scale_numeric_features()`
- ◆ `_set_categorical_dummies()`
- ◆ `_build_feature_frame()`
- ◆ `_class_to_label()`
- ◆ `get_model_status()`
- ◆ `predict_engagement()`

`_read_selected_features()`

```

def _read_selected_features() -> list[str]:
    """Read the selected feature names saved by the Task 3 pipeline."""
    features = [
        line.strip()
        for line in SELECTED_FEATURES_PATH.read_text(encoding="utf-8").splitlines()
        if line.strip()
    ]
    if not features:
        raise ValueError("selected_features.txt is empty.")
    return features

```

The `_add_engineered_features()` function recreates the Task 3 features: `WeeklyPlayLoad`, `AchievementRate`, `SessionDepth`

```

def _add_engineered_features(frame: pd.DataFrame) -> pd.DataFrame:
    """Apply the same engineered features used before model training."""
    frame = frame.copy()
    frame["WeeklyPlayLoad"] = (
        frame["SessionsPerWeek"] * frame["AvgSessionDurationMinutes"]
    )
    frame["AchievementRate"] = frame["AchievementsUnlocked"] / (
        frame["PlayerLevel"] + 1
    )
    frame["SessionDepth"] = frame["AvgSessionDurationMinutes"] / (
        frame["SessionsPerWeek"] + 1
    )
    return frame

```

The `_fit_task3_scaler()` function rebuilds the numeric scaling parameters from: `data/cleaned_gaming.csv`

```
def _fit_task3_scaler() -> None:
    """Rebuild Task 3 numeric scaling parameters from the cleaned training data."""
    global SCALER_COLUMNS, SCALER_MEANS, SCALER_SCALES, SCALER_LOAD_ERROR

    try:
        training_frame = pd.read_csv(TRAINING_DATA_PATH)
        training_frame = _add_engineered_features(training_frame)

        target_mapping = {"Low": 0, "Medium": 1, "High": 2}
        training_frame["target"] = training_frame["EngagementLevel"].map(
            target_mapping
        )

        # Match Task 3: one-hot columns are not scaled, but regular numeric
        # columns such as SessionsPerWeek and InGamePurchases are standardized.
        training_frame = pd.get_dummies(
            training_frame,
            columns=list(CATEGORICAL_COLUMNS),
            drop_first=True,
        )
        encoded_categorical_columns = [
            column
            for column in training_frame.columns
            if column.startswith(CATEGORICAL_PREFIXES)
        ]
        numeric_columns = [
            column
            for column in training_frame.select_dtypes(include=[np.number]).columns
            if column != "target" and column not in encoded_categorical_columns
        ]

        scaler = StandardScaler()
        scaler.fit(training_frame[numeric_columns])

        SCALER_COLUMNS = numeric_columns
        SCALER_MEANS = {
            column: float(mean)
            for column, mean in zip(numeric_columns, scaler.mean_, strict=True)
        }
        SCALER_SCALES = {
            column: float(scale)
            for column, scale in zip(numeric_columns, scaler.scale_, strict=True)
        }
        SCALER_LOAD_ERROR = None
    except Exception as exc:  # pragma: no cover - surfaced by /health.
        SCALER_COLUMNS = []
        SCALER_MEANS = {}
        SCALER_SCALES = {}
        SCALER_LOAD_ERROR = str(exc)
```

This is important because the training pipeline standardizes numeric columns before the model trains.

```
def _load_artifacts() -> None:
    """Load model artifacts once when this module is imported."""
    global MODEL, SELECTED_FEATURES, MODEL_LOAD_ERROR

    try:
        SELECTED_FEATURES = _read_selected_features()
        MODEL = joblib.load(MODEL_PATH)
        MODEL_LOAD_ERROR = None
    except Exception as exc:  # pragma: no cover - surfaced by /health.
        MODEL = None
        SELECTED_FEATURES = []
        MODEL_LOAD_ERROR = str(exc)

_fit_task3_scaler()

def _model_dump(player: PlayerInput) -> dict[str, Any]:
    """Return a dict from Pydantic v1 or v2 model instances."""
    if hasattr(player, "model_dump"):
        return player.model_dump()
    return player.dict()
```

```
def _scale_numeric_features(features: dict[str, Any]) -> dict[str, Any]:
    """Standardize available numeric values with the Task 3 scaler parameters."""
    scaled_features = features.copy()
    for column in SCALER_COLUMNS:
        if column not in scaled_features:
            continue

        scale = SCALER_SCALES.get(column, 1.0)
        if scale == 0:
            scaled_features[column] = 0.0
            continue

        scaled_features[column] = (
            float(scaled_features[column]) - SCALER_MEANS[column]
        ) / scale
    return scaled_features
```

The `_set_categorical_dummies()` function converts request-time categorical values into selected dummy columns such as:

GameGenre_RPG || GameGenre_Simulation || GameGenre_Strategy || Location_Europe

```
def _set_categorical_dummies(
    features: dict[str, Any],
    raw_values: dict[str, Any],
) -> dict[str, Any]:
    """Create one-hot categorical columns that match the selected feature list."""
    encoded_features = features.copy()
    for feature_name in SELECTED_FEATURES:
        for column in CATEGORICAL_COLUMNS:
            prefix = f"{column}_"
            if feature_name.startswith(prefix):
                expected_value = feature_name.removeprefix(prefix)
                encoded_features[feature_name] = int(
                    str(raw_values[column]) == expected_value
                )
    return encoded_features
```

The `_build_feature_frame()` function creates the final one-row DataFrame for prediction. It aligns the row to the exact order in `selected_features.txt` and fills missing selected features with 0.

```
def _build_feature_frame(player: PlayerInput) -> pd.DataFrame:
    """Build a single-row feature frame aligned to selected_features.txt."""
    raw_values = _model_dump(player)
    feature_frame = pd.DataFrame([raw_values])
    feature_frame = _add_engineered_features(feature_frame)

    features = feature_frame.iloc[0].to_dict()
    features = _scale_numeric_features(features)
    features = _set_categorical_dummies(features, raw_values)

    # Align request-time columns with training-time selected features. Any
    # selected feature not present in the API input is filled with 0.
    aligned_features = {
        feature_name: features.get(feature_name, 0)
        for feature_name in SELECTED_FEATURES
    }
    return pd.DataFrame([aligned_features], columns=SELECTED_FEATURES)

def _class_to_label(class_value: Any) -> str:
    """Convert model class values into API-facing labels."""
    if hasattr(class_value, "item"):
        class_value = class_value.item()
    return CLASS_LABELS.get(class_value, str(class_value))
```

The file also exposes: `get_model_status()`

This supports the `/health` endpoint by reporting whether the model, selected features, and scaler are loading correctly.

```
def get_model_status() -> dict[str, Any]:
    """Return model and preprocessing load status for the health endpoint."""
    return {
        "model_loaded": MODEL is not None,
        "selected_features_loaded": bool(SELECTED_FEATURES),
        "selected_feature_count": len(SELECTED_FEATURES),
        "scaler_loaded": not SCALER_LOAD_ERROR,
        "model_path": str(MODEL_PATH),
        "selected_features_path": str(SELECTED_FEATURES_PATH),
        "training_data_path": str(TRAINING_DATA_PATH),
        "model_error": MODEL_LOAD_ERROR,
        "scaler_error": SCALER_LOAD_ERROR,
    }
```

The main function in this file is:

```
def predict_engagement(player: PlayerInput) -> PredictionOutput:
    """Predict engagement class and probabilities for one player."""
    if MODEL is None or not SELECTED_FEATURES:
        raise RuntimeError(f"Model artifacts are not loaded: {MODEL_LOAD_ERROR}")

    feature_frame = _build_feature_frame(player)
    predicted_class = MODEL.predict(feature_frame)[0]
    probabilities = MODEL.predict_proba(feature_frame)[0]

    all_probabilities = {label: 0.0 for label in PROBABILITY_LABELS}
    for class_value, probability in zip(MODEL.classes_, probabilities, strict=True):
        class_label = _class_to_label(class_value)
        if class_label in all_probabilities:
            all_probabilities[class_label] = round(float(probability), 4)

    predicted_label = _class_to_label(predicted_class)
    confidence = all_probabilities[predicted_label]

    return PredictionOutput(
        predicted_class=predicted_label,
        confidence=confidence,
        all_probabilities=all_probabilities,
    )
```

This function:

- 1- Checks that the model and selected feature list are loaded.
- 2- Builds the aligned feature row.
- 3- Calls `MODEL.predict()`.
- 4- Calls `MODEL.predict_proba()`.
- 5- Maps numeric model classes back to Low, Medium, and High.
- 6- Rounds probabilities to 4 decimals.
- 7- Returns a `PredictionOutput` object.

api/main.py

The main.py file contains the FastAPI application and the API routes.

I create the app with:

```
app = FastAPI(title="BabiPoly Player Engagement API", version="1.0")
```

This file imports:

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware

from api.model import get_model_status, predict_engagement
from api.schemas import PlayerInput, PredictionOutput
```

It also enables CORS using CORSMiddleware, so browser-based tools and a future Streamlit frontend can call the API.

The main.py file defines three endpoints:

- ♦ GET /
- ♦ POST /predict
- ♦ GET /health

The GET / endpoint returns:

```
@app.get("/")
def read_root() -> dict[str, str]:
    """Return a simple status message confirming that the API is running."""
    return {
        "status": "ok",
        "message": "BabiPoly Engagement API is running",
    }
```

The POST /predict endpoint accepts a PlayerInput payload and returns a PredictionOutput. It uses a try/except block so model errors return an HTTP 500 response instead of crashing silently.

```
@app.post("/predict", response_model=PredictionOutput)
def predict(player: PlayerInput) -> PredictionOutput:
    """Predict the engagement level for a single player payload."""
    try:
        return predict_engagement(player)
    except Exception as exc:
        raise HTTPException(
            status_code=500,
            detail=f"Prediction failed: {exc}",
        ) from exc
```

The GET /health endpoint calls get_model_status() from api/model.py and returns the current model-loading status.

```
@app.get("/health")
def health() -> dict:
    """Return model artifact and preprocessing load status."""
    status = get_model_status()
    status["status"] = "ok" if status["model_loaded"] else "error"
    return status
```

Each endpoint includes a docstring so the code is easier to read and maintain.

api/requirements.txt

The requirements.txt file lists the packages needed to run the API:

```
fastapi
uvicorn
scikit-learn
joblib
pandas
numpy
pydantic
```

This file makes it easy to install only the backend API dependencies with:

These packages cover:

- 1- FastAPI app creation.
- 2- Uvicorn server execution.
- 3- Pydantic validation.
- 4- Pandas/Numpy feature preparation.
- 5- Joblib model loading.
- 6- Scikit-learn model prediction and scaler support.

```
PS C:\Users\Administrator> cd C:\Users\Administrator\Desktop\Internship-2025\Internship-Projects\AI\github
PS C:\Users\Administrator\Desktop\Internship-2025\Internship-Projects\AI\github> .venv\Scripts\python.exe -m pip install -r api\requirements.txt
Requirement already satisfied: fastapi in .\venv\Lib\site-packages (from -r api\requirements.txt (line 1)) (0.136.3)
Requirement already satisfied: uvicorn in .\venv\Lib\site-packages (from -r api\requirements.txt (line 2)) (0.49.0)
Requirement already satisfied: scikit-learn in .\venv\Lib\site-packages (from -r api\requirements.txt (line 3)) (1.9.0)
Requirement already satisfied: joblib in .\venv\Lib\site-packages (from -r api\requirements.txt (line 4)) (1.5.3)
Requirement already satisfied: pandas in .\venv\Lib\site-packages (from -r api\requirements.txt (line 5)) (3.0.3)
Requirement already satisfied: numpy in .\venv\Lib\site-packages (from -r api\requirements.txt (line 6)) (2.4.6)
Requirement already satisfied: pydantic in .\venv\Lib\site-packages (from -r api\requirements.txt (line 7)) (2.13.4)
Requirement already satisfied: starlette=0.46.0 in .\venv\Lib\site-packages (from fastapi->-r api\requirements.txt (line 1)) (1.2.1)
Requirement already satisfied: typing-extensions>=4.8.0 in .\venv\Lib\site-packages (from fastapi->-r api\requirements.txt (line 1)) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in .\venv\Lib\site-packages (from fastapi->-r api\requirements.txt (line 1)) (0.4.2)
Requirement already satisfied: annotated-doc>=0.0.2 in .\venv\Lib\site-packages (from fastapi->-r api\requirements.txt (line 1)) (0.0.4)
Requirement already satisfied: click>=7.0 in .\venv\Lib\site-packages (from uvicorn->-r api\requirements.txt (line 2)) (8.4.1)
Requirement already satisfied: h11>=0.8 in .\venv\Lib\site-packages (from uvicorn->-r api\requirements.txt (line 2)) (0.16.0)
Requirement already satisfied: scipy>=1.10.0 in .\venv\Lib\site-packages (from scikit-learn->-r api\requirements.txt (line 3)) (1.17.1)
Requirement already satisfied: narwhals>=2.0.1 in .\venv\Lib\site-packages (from scikit-learn->-r api\requirements.txt (line 3)) (2.22.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in .\venv\Lib\site-packages (from scikit-learn->-r api\requirements.txt (line 3)) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in .\venv\Lib\site-packages (from pandas->-r api\requirements.txt (line 5)) (2.9.0.post0)
Requirement already satisfied: tzdata in .\venv\Lib\site-packages (from pandas->-r api\requirements.txt (line 5)) (2026.2)
Requirement already satisfied: annotated-types>=0.6.0 in .\venv\Lib\site-packages (from pydantic->-r api\requirements.txt (line 7)) (0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in .\venv\Lib\site-packages (from pydantic->-r api\requirements.txt (line 7)) (2.46.4)
Requirement already satisfied: colorama in .\venv\Lib\site-packages (from click>=7.0->uvicorn->-r api\requirements.txt (line 2)) (0.4.6)
Requirement already satisfied: six>=1.5 in .\venv\Lib\site-packages (from python-dateutil>=2.8.2->pandas->-r api\requirements.txt (line 5)) (1.17.0)
Requirement already satisfied: anyio<5,>=3.6.2 in .\venv\Lib\site-packages (from starlette=0.46.0->fastapi->-r api\requirements.txt (line 1)) (4.13.0)
Requirement already satisfied: idna>=2.8 in .\venv\Lib\site-packages (from anyio<5,>=3.6.2->starlette=0.46.0->fastapi->-r api\requirements.txt (line 1)) (3.17)
```

Together, these four files create the complete REST API layer around the trained Random Forest model.

Input Schema

The request schema is called: PlayerInput

It accepts these fields:

- ♦ SessionsPerWeek
- ♦ AvgSessionDurationMinutes
- ♦ PlayerLevel
- ♦ AchievementsUnlocked
- ♦ InGamePurchases
- ♦ Gender
- ♦ Location
- ♦ GameGenre



The sample JSON payload is:

```
{
  "SessionsPerWeek": 5,
  "AvgSessionDurationMinutes": 45.0,
  "PlayerLevel": 12,
  "AchievementsUnlocked": 8,
  "InGamePurchases": 1,
  "Gender": "Male",
  "Location": "Nigeria",
  "GameGenre": "Strategy"
}
```

Example ^ Collapse all object

```
AchievementsUnlocked=8
AvgSessionDurationMinutes=45
GameGenre="Strategy"
Gender="Male"
InGamePurchases=1
Location="Nigeria"
PlayerLevel=12
SessionsPerWeek=5
```

I add validation rules:

- ♦ Session counts and numeric values must be zero or higher.
- ♦ InGamePurchases must be either 0 or 1.
- ♦ Text fields must not be empty.
- ♦ Example values are included so Swagger UI can show a useful sample request.

Output Schema

The response schema is called: PredictionOutput

It returns:

- ♦ predicted_class
- ♦ confidence
- ♦ all_probabilities

PredictionOutput ^ Collapse all object

Prediction response returned by the engagement model.

```
predicted_class* > Expand all string
confidence* > Expand all number [0, 1]
all_probabilities* > Expand all object
```

The predicted class is one of:

- ♦ Low
- ♦ Medium
- ♦ High

The confidence value is the probability of the predicted class, rounded to 4 decimals.

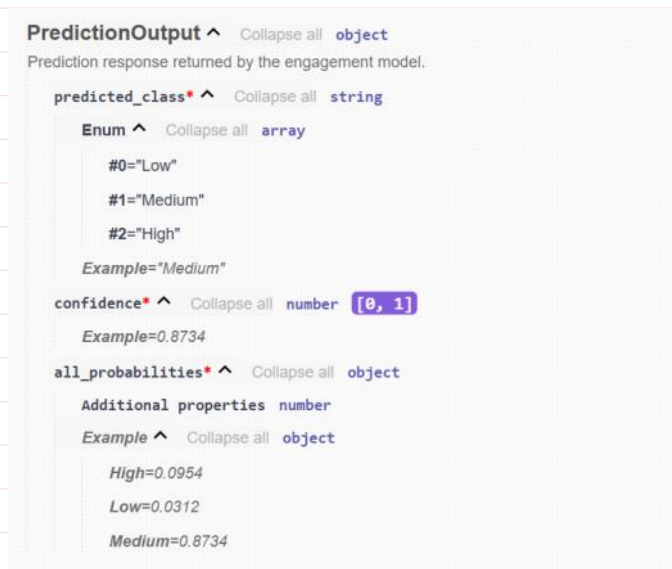
The all_probabilities dictionary always uses these keys:

- ♦ Low
- ♦ Medium

- ♦ High

Example response:

```
{  
  "predicted_class": "Low",  
  "confidence": 0.9328,  
  "all_probabilities": {  
    "Low": 0.9328,  
    "Medium": 0.0008,  
    "High": 0.0664  
  }  
}
```



Feature Engineering Logic

Inside api/model.py, I recreate the main Task 3 engineered features:

- ♦ $\text{WeeklyPlayLoad} = \text{SessionsPerWeek} * \text{AvgSessionDurationMinutes}$
- ♦ $\text{AchievementRate} = \text{AchievementsUnlocked} / (\text{PlayerLevel} + 1)$
- ♦ $\text{SessionDepth} = \text{AvgSessionDurationMinutes} / (\text{SessionsPerWeek} + 1)$

These features are important because they are part of the selected feature list and the trained model expects them.

I also rebuild the Task 3 numeric scaling parameters from: data/cleaned_gaming.csv

This keeps the request-time numeric values compatible with the training-time model input, because Task 3 standardizes numeric columns before creating X_train.csv.

Categorical Encoding Logic

The categorical input fields are:

- ♦ Gender
- ♦ Location
- ♦ GameGenre

The API converts selected categorical dummy columns into the same style used during training.

For this trained model, the selected dummy columns are:

- ♦ GameGenre_RPG
- ♦ GameGenre_Simulation
- ♦ GameGenre_Strategy
- ♦ Location_Europe

For example, if the request has:

"GameGenre": "Strategy"

Then the aligned feature row gets:

- ♦ GameGenre_Strategy = 1
- ♦ GameGenre_RPG = 0
- ♦ GameGenre_Simulation = 0

If a selected feature does not come from the API payload, the model layer fills it with 0 during alignment. This keeps the API payload focused on the required input fields while still producing a full 14-column model row.

Model Loading

The model layer loads artifacts at module startup, not inside every request.

The loaded model path is: `models/random_forest_tuned.joblib`

The loaded feature list path is: `data/selected_features.txt`

This is better for serving because the API does not reload the model for every `/predict` call. The model stays in memory after startup, and each request only builds the input feature row and runs prediction.

The model status function reports:

```
model_loaded
selected_features_loaded
selected_feature_count
scaler_loaded
model_path
selected_features_path
training_data_path
model_error
scaler_error
```

This gives the /health endpoint enough detail to debug loading problems quickly.

API Endpoints

The FastAPI app is created with:

BabiPoly Player Engagement API

1.0 OAS 3.1

[/openapi.json](#)

The API includes three endpoints.

GET /

This endpoint confirms that the API is running.

Expected response:

```
{
  "status": "ok",
  "message": "BabiPoly Engagement API is running"
}
```

GET /health

This endpoint checks whether the model artifacts and preprocessing support are loading correctly.

Expected response shape:

```
{
  "model_loaded": true,
  "selected_features_loaded": true,
```

```

"selected_feature_count": 14,
"scaler_loaded": true,
"model_path": "...",
"selected_features_path": "...",
"training_data_path": "...",
"model_error": null,
"scaler_error": null,
"status": "ok"
}

```

POST /predict

This endpoint accepts a PlayerInput JSON payload and returns a PredictionOutput.

It wraps the prediction call in try/except. If the model fails, the endpoint raises a 500 HTTP error with the prediction error detail.

CORS Setup

I enable CORS for all origins in api/main.py:

```

allow_origins=["*"]
allow_methods=["*"]
allow_headers=["*"]

```

This supports local testing and also prepares the API for a possible Streamlit frontend in the next step.

Test Results

```

(.venv) PS C:\Users\Administrator\Desktop\Internship-2025\Internship-Projects\AI\github> .venv\Scripts\python.exe -m uvicorn api.main:app --reload --port 8000
+-----+
+ [32mINFO+ [0m: Will watch for changes in these directories: ['C:\Users\Administrator\Desktop\Internship-2025\Internship-Projects\AI\github']
+ [32mINFO+ [0m: Uvicorn running on +[1mhttp://127.0.0.1:8000+ [0m (Press CTRL+C to quit)
+ [32mINFO+ [0m: Started reloader process [+ [36m-[1m9340+ [0m] using [+ [36m-[1mStatReload+ [0m]
+ [32mINFO+ [0m: Started server process [+ [36m9928+ [0m]
+ [32mINFO+ [0m: Waiting for application startup.
+ [32mINFO+ [0m: Application startup complete.
+ [32mINFO+ [0m: 127.0.0.1:62140 - "+[1mGET /docs HTTP/1.1+ [0m" +[32m200 OK+ [0m
+ [32mINFO+ [0m: 127.0.0.1:62140 - "+[1mGET /openapi.json HTTP/1.1+ [0m" +[32m200 OK+ [0m
+ [32mINFO+ [0m: 127.0.0.1:62141 - "+[1mGET /docs HTTP/1.1+ [0m" +[32m200 OK+ [0m
+ [32mINFO+ [0m: 127.0.0.1:62141 - "+[1mGET /openapi.json HTTP/1.1+ [0m" +[32m200 OK+ [0m
+ [32mINFO+ [0m: 127.0.0.1:63182 - "+[1mGET /openapi.json HTTP/1.1+ [0m" +[32m200 OK+ [0m
+ [32mINFO+ [0m: 127.0.0.1:63182 - "+[1mGET /favicon.ico HTTP/1.1+ [0m" +[31m404 Not Found+ [0m
+ [32mINFO+ [0m: 127.0.0.1:63209 - "+[1mGET /health HTTP/1.1+ [0m" +[32m200 OK+ [0m
+ [32mINFO+ [0m: 127.0.0.1:63233 - "+[1mGET / HTTP/1.1+ [0m" +[32m200 OK+ [0m
+-----+

```


BabiPoly Player Engagement API

1.0 OAS 3.1

/openapi.json

default

GET / Read Root

POST /predict Predict

GET /health Health

GET:

Curl

```
curl -X 'GET' \
  'http://localhost:8000/' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8000/

Server response

Code Details

200

Response body

```
{
  "status": "ok",
  "message": "BabiPoly Engagement API is running"
}
```

Health:

200

Response body

```
{
  "model_loaded": true,
  "selected_features_loaded": true,
  "selected_feature_count": 14,
  "scaler_loaded": true,
  "model_path": "C:\\Users\\Administrator\\Desktop\\Internship-2\\BabiPoly\\model.pkl",
  "selected_features_path": "C:\\Users\\Administrator\\Desktop\\Internship-2\\BabiPoly\\selected_features.pkl",
  "training_data_path": "C:\\Users\\Administrator\\Desktop\\Internship-2\\BabiPoly\\training_data.pkl",
  "model_error": null,
  "scaler_error": null,
  "status": "ok"
}
```

Predict:

1- Low

Edit Value | Schema

```
{
  "AchievementsUnlocked": 8,
  "AvgSessionDurationMinutes": 45,
  "GameGenre": "Strategy",
  "Gender": "Male",
  "InGamePurchases": 1,
  "Location": "Nigeria",
  "PlayerLevel": 12,
  "SessionsPerWeek": 5
}
```

200

Response body

```
{
  "predicted_class": "Low",
  "confidence": 0.9328,
  "all_probabilities": {
    "Low": 0.9328,
    "Medium": 0.0664,
    "High": 0.0008
  }
}
```

2- Low

```
{
  "AchievementsUnlocked": 14,
  "AvgSessionDurationMinutes": 85,
  "GameGenre": "Strategy",
  "Gender": "Male",
  "InGamePurchases": 1,
  "Location": "Nigeria",
  "PlayerLevel": 15,
  "SessionsPerWeek": 5
}
```

```
{
  "predicted_class": "Medium",
  "confidence": 0.9329,
  "all_probabilities": {
    "Low": 0.0237,
    "Medium": 0.9329,
    "High": 0.0434
  }
}
```

3- High

```
{
  "AchievementsUnlocked": 20,
  "AvgSessionDurationMinutes": 125,
  "GameGenre": "Strategy",
  "Gender": "Male",
  "InGamePurchases": 1,
  "Location": "Nigeria",
  "PlayerLevel": 25,
  "SessionsPerWeek": 14
}
```

Response body

```
{  
  "predicted_class": "High",  
  "confidence": 0.9304,  
  "all_probabilities": {  
    "Low": 0.031,  
    "Medium": 0.0385,  
    "High": 0.9304  
  }  
}
```

This confirms that the API can load the trained model, accept JSON input, prepare the features, and return a valid prediction response.

What This Task Adds to the Project

This task turns the trained Random Forest model into a usable backend service.

Before this task, the model exists as a local .joblib artifact. After this task, the model can receive live JSON requests through a REST API and return clean prediction results that another app, dashboard, or frontend can use.

The important progress is:

- 1- The trained model is no longer only used from notebooks/scripts.
- 2- The input and output contracts are documented through Pydantic and Swagger UI.
- 3- The preprocessing logic is centralized in api/model.py.
- 4- The API exposes health checks for debugging.
- 5- The API is ready for a Streamlit or web frontend connection.